

PROJET TUTEURÉ N°2

RÉALISATION D'UNE APPLICATION MOBILE

Développement Android moderne avec Jetpack Compose

Étude de cas : conception d'une application Calculatrice

Informations module

Intitulé : Projet tutoré N°2 — Réalisation d'une application mobile

Code de l'UE : B2COM0602

Niveau : Bachelor 2 (B2) — IT

Chargé du cours : Ing. BODJONA Bataka Pignanti

Édition : 2026 · Kotlin · Compose BOM 2026.05 · Material 3

Support de cours et de travaux pratiques — Niveau débutant → avancé

Table des matières

Avant-propos.....	3
Partie liminaire — Le développement mobile & le projet tuteuré	5
Partie 0 — Mise en route & installation (Windows/macOS)	9
Partie AS — Android Studio en profondeur	15
Partie AND — Concepts fondamentaux d'Android	23
Partie 1 — Kotlin essentiel pour Compose.....	27
Partie 2 — Les fondamentaux de Compose	32
Partie 3 — Architecture & navigation.....	41
Partie 4 — UI avancée	47
Partie 5 — Données & tests.....	52
Partie 6 — Bonnes pratiques & projet fil rouge	58
Partie 7 — Étude de cas : application Calculatrice	60
Partie 8 — Gestion de projet, qualité & UX	66
Partie 9 — Le développement multiplateforme.....	69
Partie 10 — Déploiement & publication	71
Partie 11 — Le projet tuteuré : organisation & évaluation.....	74
Partie 12 — Étude de cas guidée : application To-Do	77
Partie 13 — Sécurité, données personnelles & bonnes pratiques	80
Partie 14 — Accessibilité & internationalisation.....	82
Partie 15 — Performance & optimisation	84
Partie 16 — Bonnes pratiques d'ingénierie & qualité du code	86
Partie 17 — Étude de cas guidée : application Météo (API REST)	88
Partie 18 — Recettes pratiques (FAQ technique).....	91
Partie 19 — Conception d'interface : maquettage & Material Design	93
Partie 20 — Synthèse & fiches récapitulatives.....	95
Annexes (aide-mémoire, erreurs fréquentes, ressources, modèles)	97

Avant-propos

Ce support de cours est une initiation complète et progressive au développement d'applications Android avec **Jetpack Compose**, la boîte à outils moderne d'interface utilisateur recommandée par Google. Il s'adresse aussi bien au débutant qui découvre le développement mobile qu'au développeur expérimenté souhaitant consolider sa maîtrise de Compose.

Le parcours est organisé en six parties qui s'enchaînent logiquement : on part des fondamentaux du langage et des composables, puis on construit une architecture solide, on enrichit l'interface, et enfin on relie l'application à des données réelles tout en la testant.

Comment l'utiliser : chaque chapitre alterne théorie, exemples de code commentés et exercices. Tapez le code vous-même plutôt que de le copier : c'est en se trompant que l'on apprend le plus vite. Les encadrés verts donnent des conseils, les encadrés orange signalent les pièges fréquents.

Prérequis

Des bases de programmation (variables, fonctions, conditions, boucles) dans n'importe quel langage suffisent pour démarrer la Partie 1. Aucune connaissance préalable d'Android n'est requise.

Partie	Contenu	Niveau
0 · Mise en route	Paradigme déclaratif, outils, configuration Gradle	Débutant
1 · Kotlin essentiel	Le minimum de Kotlin pour être à l'aise en Compose	Débutant
2 · Fondamentaux Compose	Composables, layouts, modifieurs, état, effets	Déb. → Inter.
3 · Architecture & navigation	MVVM, ViewModel, StateFlow, Navigation, Hilt	Intermédiaire
4 · UI avancée	Material 3, animations, layouts custom, performance	Inter. → Avancé
5 · Données & tests	Coroutines/Flow, Retrofit, Room, tests UI & unitaires	Avancé

Comment utiliser ce support

Ce document est à la fois un **cours**, un **guide de travaux pratiques** et un **manuel de référence** pour le projet. Plusieurs parcours de lecture sont possibles :

- **Lecture linéaire** (recommandée pour débiter) : suivez les parties dans l'ordre, en réalisant les exercices.
- **En atelier** : appuyez-vous sur les parties « installation », « Android Studio » et les études de cas pour les séances pratiques.
- **En référence** : pendant le projet, piochez dans les recettes (Partie 18), les annexes (modèles, commandes) et le glossaire.

Les **encadrés verts** donnent des conseils et points clés ; les **encadrés orange** signalent les pièges fréquents. Les blocs de code sont en police à chasse fixe, prêts à être saisis et adaptés.

Apprendre en faisant

Tapez le code vous-même, exécutez-le, modifiez-le, observez ce qui change. La compréhension vient de la pratique et de l'erreur, bien plus que de la lecture passive. Les trois études de cas (calculatrice, To-Do, météo) sont des points de départ concrets pour votre projet.

Partie liminaire — Le développement mobile et le projet tuteuré

Avant d'entrer dans la technique, cette partie pose le **cadre** du module : pourquoi le développement mobile est devenu une compétence essentielle, quelles sont les grandes familles de technologies, en quoi consiste le cycle de vie complet d'une application, et ce que ce projet tuteuré attend concrètement de vous. C'est la boussole qui donne du sens à tout le reste du support.

L.1 Le développement mobile, une compétence clé

Le développement d'applications mobiles est aujourd'hui une **compétence clé pour tout informaticien**. Le smartphone est devenu le premier point d'accès au numérique pour des milliards de personnes : commerce, banque, santé, éducation, transport, divertissement — presque chaque service possède désormais son application. Savoir concevoir, développer et livrer une application mobile de qualité est donc une aptitude recherchée, quel que soit le métier visé (développeur, chef de projet, analyste, entrepreneur).

Mais « savoir coder un écran » ne suffit pas. Une application professionnelle suppose de maîtriser **l'ensemble du cycle de vie** : comprendre un besoin, le traduire en spécifications, concevoir une architecture, écrire un code propre et testé, déployer sur les magasins d'applications, puis assurer le suivi (corrections, mises à jour, mesure d'usage). Ce sont ces savoir-faire que le présent module vise à développer.

Ce que vous saurez faire à l'issue du module

Analyser un besoin et rédiger un cahier des charges ; concevoir et développer une application Android moderne en Kotlin/Jetpack Compose ; organiser et tester le code ; générer un livrable installable (APK) ; travailler en équipe et documenter votre travail comme en situation professionnelle.

L.2 Applications natives et multiplateformes

On distingue deux grandes approches pour construire une application mobile, chacune avec ses forces.

Le développement natif

Une application **native** est développée spécifiquement pour une plateforme, avec ses outils et langages officiels : **Android** (Kotlin / Jetpack Compose) et **iOS** (Swift / SwiftUI). Elle offre les meilleures performances, un accès complet aux capacités de l'appareil (capteurs, caméra, notifications) et une expérience parfaitement intégrée au système. Son inconvénient : il faut écrire (et maintenir) **deux bases de code** pour couvrir Android et iOS.

Le développement multiplateforme

Les frameworks **multiplateformes** permettent d'écrire un code largement **partagé** qui s'exécute sur plusieurs systèmes. Les plus répandus :

Technologie	Langage	Principe	Points forts
Android natif	Kotlin	Jetpack Compose	Performances, intégration système, accès matériel complet
iOS natif	Swift	SwiftUI	Idem pour l'écosystème Apple
Flutter	Dart	Moteur de rendu propre (Skia/Impeller)	Une seule UI pour Android/iOS/web/desktop, rendu fluide
React Native	JavaScript/TS	Composants natifs pilotés par JS	Réutilise l'écosystème React, large communauté
Kotlin Multiplatform	Kotlin	Logique partagée + UI native ou Compose MP	Partage la logique métier, UI native possible

Quel choix pour ce module ?

Nous apprenons le **développement Android natif avec Kotlin et Jetpack Compose**. C'est la base la plus formatrice : elle fait comprendre en profondeur le fonctionnement d'une plateforme mobile (cycle de vie, UI déclarative, état, threads), des concepts qui se transposent ensuite à Flutter, React Native ou iOS. La Partie « Développement multiplateforme » compare ces approches plus loin.

L.3 Le cycle de vie d'un projet d'application mobile

Réaliser une application, ce n'est pas seulement programmer. C'est suivre un **cycle de vie** en plusieurs phases, chacune produisant des livrables. Comprendre ce cycle est l'un des principaux objectifs de ce projet tuteuré.

Phase	Objectif	Livrables typiques
1 · Analyse des besoins	Comprendre le problème, les utilisateurs, les contraintes	Cahier des charges, user stories
2 · Conception	Définir l'architecture, les écrans, le modèle de données	Maquettes (wireframes), schéma d'architecture
3 · Développement	Implémenter les fonctionnalités	Code source organisé, dépôt Git
4 · Tests	Vérifier la qualité et la conformité	Tests unitaires & d'UI, rapport de bugs
5 · Déploiement	Produire et publier l'application	APK/AAB signé, fiche Play Store
6 · Suivi (maintenance)	Corriger, améliorer, mesurer l'usage	Mises à jour, suivi des plantages et analytics

Un processus itératif

Ces phases ne sont pas strictement linéaires : on revient souvent en arrière (un test révèle un défaut de conception, un retour utilisateur fait évoluer le besoin). Les méthodes agiles, vues plus loin,

organisent ce va-et-vient en cycles courts. L'essentiel : à chaque étape, on produit quelque chose de concret et vérifiable.

L.4 Objectifs pédagogiques du projet tuteuré

Ce projet tuteuré, **second de la série**, vise à vous mettre en **situation professionnelle**. L'objectif n'est pas seulement de produire du code qui fonctionne, mais d'apprendre à conduire un projet logiciel de bout en bout, comme dans une véritable équipe de développement.

Concrètement, le projet poursuit les objectifs suivants :

- **Maîtriser le cycle de vie complet** d'une application : de l'analyse des besoins au suivi, en passant par la conception, le développement, les tests et le déploiement.
- **Développer l'autonomie** : savoir chercher l'information (documentation officielle, messages d'erreur), expérimenter et résoudre les problèmes sans dépendre en permanence de l'enseignant.
- **Apprendre la gestion de projet** : planifier, répartir les tâches, suivre l'avancement, respecter des jalons.
- **Soigner la qualité du code** : lisibilité, architecture claire, tests, contrôle de version.
- **Penser l'expérience utilisateur (UX)** : concevoir des écrans clairs, ergonomiques et accessibles.
- **Produire de la documentation** : expliquer son travail (README, manuel, rapport) pour qu'il soit compréhensible et réutilisable.

L.5 Organisation : un travail d'équipe

Le projet se réalise **par groupes de trois ou quatre étudiants**. Chaque équipe doit réaliser une **application mobile fonctionnelle** répondant à un cahier des charges défini — soit **imposé par l'enseignant**, soit **proposé par l'équipe puis validé** par l'enseignant.

Le travail en équipe est une compétence à part entière. Pour être efficace, répartissez les **rôles** (tout en restant solidaires) :

Rôle	Responsabilités principales
Chef de projet / Scrum master	Planifie, anime les points d'équipe, suit les jalons, fait le lien avec l'enseignant
Responsable conception / UX	Maquettes, parcours utilisateur, cohérence visuelle (thème, composants)
Responsable développement	Architecture du code, intégration, revue des contributions
Responsable qualité / tests	Tests, suivi des bugs, documentation, préparation de la démo

Les rôles tournent

Dans un groupe de trois ou quatre, chacun code : les rôles ci-dessus désignent un référent, pas une exclusivité. L'important est que chaque responsabilité ait un propriétaire clairement identifié, et que tout le monde participe au développement.

L.6 Critères de réussite

L'accent est mis sur l'**autonomie**, la **gestion de projet**, la **qualité du code**, l'**expérience utilisateur** et la **documentation**. Voici les attentes traduites en critères concrets, qui structureront l'évaluation finale (détaillée dans la dernière partie).

Critère	Ce qui est attendu
Fonctionnalité	L'application répond au cahier des charges et fonctionne sans plantage
Qualité du code	Code lisible, architecture claire (UI/ViewModel/Données), versionné sous Git
Expérience utilisateur	Interface ergonomique, cohérente, accessible, agréable
Tests	Présence de tests pertinents (logique et écrans clés)
Gestion de projet	Planning tenu, jalons respectés, répartition claire des tâches
Documentation	README, manuel d'utilisation, rapport de projet
Présentation	Démonstration claire et maîtrisée du produit final

Fil conducteur du support

Les parties suivantes vous donnent tous les outils techniques pour réussir ce projet : installer et maîtriser l'environnement (Android Studio), apprendre Kotlin et Compose, structurer le code, gérer les données, tester, puis déployer. La dernière partie revient sur l'organisation concrète du projet de groupe et son évaluation.

Partie 0 — Mise en route

0.1 Pourquoi Jetpack Compose ?

Pendant plus de dix ans, l'interface Android s'est écrite en **XML** (les fichiers de layout) manipulé depuis du code Java/Kotlin via le système des **View**. Ce modèle est dit **impératif** : on décrit chaque étape pour modifier l'écran (« trouve la vue, change son texte, masque ce bouton »). Il est verbeux, source de bugs (états désynchronisés entre le XML et le code) et difficile à faire évoluer.

Jetpack Compose adopte une approche **déclarative** : on décrit à quoi l'écran doit ressembler **pour un état donné**, et le framework se charge de mettre à jour l'affichage quand cet état change. L'UI devient une simple fonction de l'état : **UI = f(état)**.

Critère	Système View / XML (ancien)	Jetpack Compose (moderne)
Paradigme	Impératif	Déclaratif
Langage UI	XML + Kotlin	100 % Kotlin
Mise à jour UI	Manuelle (findViewById, setText...)	Automatique (recomposition)
Réutilisation	Custom views, includes	Fonctions composables
Aperçu	Éditeur de layout	@Preview en temps réel
Verbo­sité	Élevée	Faible

À retenir

Compose ne remplace pas seulement le XML : il change la façon de penser l'UI. On ne « modifie » plus l'écran, on le « redéclare » à chaque changement d'état. Le framework calcule la différence et met à jour l'affichage de façon optimale.

0.2 L'écosystème et les outils

Pour développer en Compose, vous aurez besoin de :

- **Android Studio** (version la plus récente) — l'IDE officiel, qui intègre l'éditeur, l'émulateur et les aperçus Compose.
- **Kotlin** — Compose est écrit en Kotlin et n'est utilisable qu'avec ce langage. La version 2.x du compilateur intègre directement le plugin Compose.
- **Le Compose BOM** (Bill of Materials) — un catalogue de versions qui garantit la compatibilité entre toutes les bibliothèques Compose. En 2026, la version stable est **2026.05.x** (artefacts core 1.11.x, Material 3 1.4.x).
- **Le JDK 17+** et le SDK Android (API 24 minimum recommandé, compilation sur l'API la plus récente).

0.3 Configurer un projet Compose (Gradle)

Un projet Compose moderne utilise le DSL Kotlin de Gradle et le BOM. Voici l'essentiel du fichier de module `build.gradle.kts` :

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    // Plugin du compilateur Compose (Kotlin 2.0+)
    id("org.jetbrains.kotlin.plugin.compose")
}

android {
    namespace = "com.exemple.monapp"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.exemple.monapp"
        minSdk = 24
        targetSdk = 35
    }

    buildFeatures { compose = true } // active Compose

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions { jvmTarget = "17" }
}

dependencies {
    // Le BOM aligne automatiquement toutes les versions Compose
    val composeBom = platform("androidx.compose:compose-bom:2026.05.00")
    implementation(composeBom)

    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.ui:ui-tooling-preview")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.activity:activity-compose")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose")

    debugImplementation("androidx.compose.ui:ui-tooling")
    androidTestImplementation(composeBom)
    androidTestImplementation("androidx.compose.ui:ui-test-junit4")
    debugImplementation("androidx.compose.ui:ui-test-manifest")
}
```

Le rôle du BOM

Grâce au BOM, vous ne précisez **aucun numéro de version** sur les artefacts Compose : il les fixe pour vous de façon cohérente. Pour mettre à jour Compose, il suffit de changer une seule ligne — la version du BOM.

0.4 Le point d'entrée : une Activity

Une application Compose démarre toujours dans une [Activity](#) qui appelle `setContent { ... }`. Tout ce qui se trouve dans ce bloc est de l'UI Compose.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MonAppTheme {           // thème Material 3 de l'app
                Surface {
                    Greeting("Android")
                }
            }
        }
    }
}

@Composable
fun Greeting(nom: String) {
    Text(text = "Bonjour $nom !")
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MonAppTheme { Greeting("Compose") }
}
```

La fonction annotée `@Preview` s'affiche directement dans Android Studio, sans lancer l'émulateur : c'est l'un des grands atouts de Compose pour itérer vite.

0.5 Installer Android Studio sur Windows

Configuration recommandée : Windows 10/11 64 bits, 8 Go de RAM minimum (16 Go conseillés), 8 Go d'espace disque libre, et la virtualisation matérielle activée dans le BIOS (Intel VT-x / AMD-V) pour l'émulateur.

1. Téléchargez Android Studio depuis le site officiel : developer.android.com/studio. Le fichier est un installeur .exe.
2. Lancez l'installeur et acceptez les valeurs par défaut. Cochez « Android Virtual Device » pour installer l'émulateur.
3. Au premier démarrage, choisissez l'installation « Standard » : Android Studio télécharge le SDK Android, les outils de build et une image système pour l'émulateur (prévoir une bonne connexion).
4. Acceptez les licences du SDK lorsqu'elles s'affichent, puis laissez le téléchargement se terminer.
5. Vérifiez l'installation du JDK : Android Studio embarque son propre JDK (JBR). Aucune installation Java séparée n'est nécessaire.

Accélération de l'émulateur (Windows)

Pour un émulateur fluide, activez **Windows Hypervisor Platform** (fonctionnalités Windows) ou installez le pilote **Intel HAXM** proposé par Android Studio. Sans accélération, l'émulateur sera très lent ; à défaut, branchez un téléphone physique en mode développeur.

0.6 Installer Android Studio sur macOS

Configuration recommandée : macOS 12 (Monterey) ou plus récent, puces Apple Silicon (M1/M2/M3/M4) ou Intel, 8 Go de RAM minimum (16 Go conseillés), 8 Go d'espace libre.

6. Téléchargez le fichier .dmg sur developer.android.com/studio. Le site détecte automatiquement votre processeur (Apple Silicon ou Intel) — choisissez la bonne version.
7. Ouvrez le .dmg, puis glissez l'icône **Android Studio** dans le dossier **Applications**.
8. Lancez Android Studio depuis le Launchpad. À la première ouverture, macOS demande une confirmation de sécurité : autorisez l'application.
9. Choisissez l'installation « Standard » : le SDK Android, les build-tools et une image système d'émulateur sont téléchargés automatiquement.
10. Acceptez les licences du SDK pour finaliser.

Émulateur sur Apple Silicon

Sur Mac M1/M2/M3, choisissez une image système d'émulateur de type **arm64-v8a** : elle est nativement accélérée et très fluide. L'émulateur x86 n'est pas adapté à Apple Silicon.

0.7 Premier lancement : projet, émulateur et appareil

Une fois l'IDE prêt, créez et exécutez votre première application :

11. Écran d'accueil → **New Project** → modèle **Empty Activity** (il utilise Jetpack Compose par défaut).
12. Renseignez le **Name**, le **Package name** (ex. com.b2com.monapp), le dossier, le langage **Kotlin** et le **Minimum SDK** (API 24).

13. Laissez Gradle se synchroniser (barre de progression en bas). À la fin, l'arborescence du projet apparaît.
14. Créez un émulateur : **Device Manager** → **Create Device** → choisissez un téléphone (ex. Pixel 7) → une image système (API récente) → **Finish**.
15. Sélectionnez l'appareil dans la barre du haut, puis cliquez sur **Run** . L'application se compile et se lance sur l'émulateur.
16. Pour un **appareil physique** : activez les [Options pour les développeurs](#) (tapez 7 fois sur « Numéro de build »), activez le [Débogage USB](#), branchez le téléphone et autorisez la connexion.

Anatomie d'un projet Android

Les dossiers clés : [app/src/main/java](#) (code Kotlin), [app/src/main/res](#) (ressources : icônes, chaînes, thèmes), [AndroidManifest.xml](#) (déclaration de l'app et des écrans), et les fichiers [build.gradle.kts](#) (configuration de build et dépendances).

0.8 Installer Git et préparer le travail d'équipe

Le projet se réalisant en groupe, **Git** est indispensable dès le départ. Préparez l'environnement collaboratif :

17. Installer Git : **git-scm.com** (Windows/macOS). Sur macOS, il s'installe aussi avec les outils en ligne de commande de Xcode.
18. Créer un compte sur **GitHub** ou **GitLab** et un dépôt pour l'équipe.
19. Configurer son identité : `git config --global user.name "Prénom Nom"` et `git config --global user.email "..."`.
20. Dans Android Studio : **VCS** → **Share Project on GitHub** pour publier le projet, ou cloner le dépôt existant de l'équipe.
21. Ajouter un [.gitignore](#) Android (Android Studio en propose un) pour exclure [build/](#), [.idea/](#) et fichiers locaux.

Un dépôt par projet, dès le premier jour

Créez le dépôt avant d'écrire la première ligne. Chaque membre clone, code sur sa branche, et pousse régulièrement. Un historique riche et partagé est à la fois un outil de travail et une preuve d'implication de chacun.

0.9 Dépannage de l'installation

Problème	Solution
« SDK location not found »	Ouvrir le SDK Manager et installer une plateforme + les build-tools
Téléchargement Gradle très long (1re fois)	Normal : laisser finir ; conserver la connexion
L'émulateur refuse de démarrer	Vérifier l'accélération matérielle ; réinstaller l'image système
« Minimum supported Gradle version »	Mettre à jour le plugin Android Gradle / le wrapper

Problème	Solution
Pas de périphérique détecté (USB)	Activer le débogage USB et accepter l'autorisation sur le téléphone
Android Studio lent	Augmenter la mémoire (Help → Change Memory Settings)

La première compilation est la plus longue

Au premier lancement, Gradle télécharge de nombreuses dépendances : c'est normal que ce soit long. Les compilations suivantes sont bien plus rapides grâce au cache. Ne l'interrompez pas.

Partie AS — Android Studio en profondeur

Android Studio est l'**environnement de développement intégré (IDE)** officiel pour Android, bâti sur IntelliJ IDEA de JetBrains. C'est bien plus qu'un éditeur de texte : il réunit l'écriture du code, la compilation, le débogage, le profilage des performances, la gestion des appareils virtuels et l'inspection de l'application en un seul outil. Bien le connaître fait gagner énormément de temps. Cette partie passe en revue ses principaux composants, un par un.

Pourquoi ce chapitre est important

Beaucoup d'étudiants n'utilisent que 10 % de l'IDE et perdent du temps sur des tâches qu'Android Studio automatise. Maîtriser l'éditeur, le débogueur, Logcat et l'émulateur transforme votre productivité et votre capacité à diagnostiquer les problèmes en autonomie — exactement ce que vise ce projet.

AS.1 Vue d'ensemble de la fenêtre principale

La fenêtre d'Android Studio s'organise en zones. Savoir les nommer aide à suivre les instructions et à se repérer :

Zone	Rôle
Barre de menus & barre d'outils	Commandes globales et boutons Run  , Debug, SDK Manager, etc.
Barre de navigation	Chemin du fichier courant ; sélecteur d'appareil et de configuration de lancement
Fenêtre de projet (gauche)	Arborescence des fichiers (vue Android ou Project)
Éditeur (centre)	Édition du code, avec onglets et aperçu Compose (Split)
Fenêtres d'outils (bords)	Logcat, Build, Run, Terminal, Git, Profiler... ouvrables par onglets
Barre d'état (bas)	Avancement des tâches Gradle, position du curseur, branche Git

Astuce : tout trouver avec « Search Everywhere »

Appuyez **deux fois sur Maj** (Shift Shift) pour ouvrir « Search Everywhere » : on y cherche un fichier, une classe, une action de menu ou un réglage. C'est le raccourci le plus utile de l'IDE.

AS.2 La structure d'un projet

La fenêtre de projet propose plusieurs **vues**. La vue **Android** (par défaut) regroupe logiquement les fichiers utiles ; la vue **Project** montre l'arborescence réelle sur le disque. Voici les éléments essentiels :

Élément	Description
manifests/AndroidManifest.xml	Déclare l'application, ses écrans (activities), permissions et icône
java/ (ou kotlin+java)	Code source Kotlin, organisé par packages

Élément	Description
res/	Ressources : drawable (images), values (chaînes, couleurs, thèmes), mipmap (icônes)
res/values/strings.xml	Toutes les chaînes de texte (pour la traduction)
Gradle Scripts/build.gradle.kts	Configuration de build : SDK, dépendances, options
gradle/libs.versions.toml	Catalogue de versions des dépendances (version catalog)
gradle.properties	Réglages globaux de build (mémoire, options AndroidX)

Vue Android vs vue Project

Si vous ne retrouvez pas un dossier (par ex. [gradle/](#) ou les fichiers générés), basculez la liste déroulante en haut de la fenêtre de projet sur [Project](#) : vous verrez la structure réelle du disque.

AS.3 L'éditeur de code intelligent

L'éditeur est le cœur de l'IDE. Il comprend le code et accélère l'écriture grâce à de nombreuses aides :

- **Complétion intelligente** : propose noms, méthodes et imports au fur et à mesure de la frappe (Ctrl/Cmd + Espace pour forcer).
- **Imports automatiques** : ajoute l'import manquant (Alt/⌘ + Entrée sur un symbole souligné).
- **Refactoring** : renommer (Shift + F6), extraire une fonction/variable, déplacer — l'IDE met à jour toutes les références.
- **Navigation** : aller à la déclaration (Ctrl/Cmd + clic), rechercher les usages (Alt/⌘ + F7), structure du fichier (Ctrl/Cmd + F12).
- **Analyse en direct** : souligne erreurs et avertissements ; l'ampoule propose des corrections rapides (quick-fixes).
- **Reformatage** : Ctrl/Cmd + Alt/⌘ + L applique le style de code Kotlin officiel.

Lire les soulignements

Rouge = erreur de compilation (à corriger). Jaune/gris = avertissement ou suggestion. Survolez le soulignement pour la description, et placez le curseur dessus puis Alt/⌘ + Entrée pour voir les corrections proposées.

AS.4 Le système de build Gradle

Android Studio compile votre application avec **Gradle**, un puissant système de build. Vous le configurez via des fichiers [build.gradle.kts](#) (langage Kotlin). Comprendre Gradle évite bien des blocages.

Les fichiers de build

Fichier	Rôle
settings.gradle.kts	Liste les modules du projet et les dépôts de plugins
build.gradle.kts (racine)	Déclare les plugins et versions communs

Fichier	Rôle
app/build.gradle.kts	Configure le module application : SDK, dépendances, build types
gradle/libs.versions.toml	Centralise les versions (version catalog)

Concepts clés

- **Dépendances** : bibliothèques externes ajoutées dans le bloc `dependencies { }`. `implementation` pour l'usage interne, `api` pour exposer, `testImplementation` pour les tests.
- **Build types** : `debug` (développement) et `release` (publication, optimisé/minifié).
- **Build variants** : combinaisons de build types et de « product flavors » (ex. version gratuite/payante).
- **BOM** : aligne les versions d'un ensemble de bibliothèques (ex. Compose BOM).
- **Sync** : après toute modification d'un fichier Gradle, cliquez « Sync Now » pour que l'IDE prenne en compte les changements.

```
// Extrait : déclarer une dépendance et un build type
android {
    buildTypes {
        release {
            isMinifyEnabled = true           // réduit et obfusque le code
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
}
dependencies {
    implementation("androidx.core:core-ktx:1.13.1") // bibliothèque
    testImplementation("junit:junit:4.13.2")        // tests unitaires
}
```

Quand le build échoue

Lisez la fenêtre **Build** en bas : le message d'erreur indique presque toujours la cause (dépendance introuvable, version incompatible, SDK manquant). En dernier recours : **File** → **Invalidate Caches** / **Restart**, ou **Build** → **Clean Project** puis **Rebuild Project**.

AS.5 Le SDK Manager

Le **SDK Manager** (icône dédiée ou **Settings** → **Languages & Frameworks** → **Android SDK**) gère les composants du kit de développement Android :

- **SDK Platforms** : les versions d'Android (API levels) sur lesquelles compiler et tester.
- **SDK Tools** : outils de build (Android SDK Build-Tools), l'émulateur, les images système, le NDK si besoin.
- **Sources** : le code source d'Android (utile pour explorer les classes du framework).

compileSdk, targetSdk, minSdk

minSdk = version Android minimale supportée (ex. 24). **targetSdk** = version pour laquelle l'app est testée et optimisée. **compileSdk** = version du SDK utilisée pour compiler (la plus récente, en général).

AS.6 Le Device Manager et l'émulateur

Le **Device Manager** crée et gère des **appareils virtuels (AVD)** — des téléphones/tablettes émuls sur votre ordinateur — et liste les appareils physiques connectés.

- Créer un AVD : choisir un modèle (Pixel...), une image système (API + architecture), un nom.
- Sur Apple Silicon (M1/M2/M3), préférez une image **arm64-v8a** ; sur PC, **x86_64** avec accélération matérielle.
- **Snapshots** : l'émulateur sauvegarde son état pour démarrer plus vite.
- **Contrôles étendus** (Extended Controls) : simuler position GPS, appels/SMS, batterie, rotation, capteurs, empreinte digitale.

Émulateur lent ?

Activez l'accélération matérielle (Hyper-V/WHXP sur Windows, HVM sur macOS), allouez assez de RAM à l'AVD, ou testez directement sur un téléphone physique en débogage USB — souvent plus rapide.

AS.7 Exécuter et déboguer une application

Le bouton **Run**  compile, installe et lance l'application sur l'appareil sélectionné. Le bouton **Debug** fait de même en attachant le **débogueur**, qui permet d'arrêter l'exécution et d'inspecter le programme.

Utiliser les points d'arrêt (breakpoints)

- Cliquez dans la marge à gauche d'une ligne pour poser un **point d'arrêt** (point rouge).
- En mode Debug, l'exécution s'arrête à cette ligne : vous voyez la valeur des variables.
- **Step Over** (F8) exécute la ligne ; **Step Into** (F7) entre dans la fonction appelée ; **Resume** continue.
- **Evaluate Expression** : évaluer une expression à la volée pendant l'arrêt.
- **Watches** : surveiller en permanence certaines variables.

Déboguer, c'est observer

Plutôt que d'ajouter des dizaines de logs, posez un point d'arrêt et inspectez l'état réel du programme. Vous comprendrez bien plus vite pourquoi une valeur est inattendue.

AS.8 Logcat : lire les journaux

La fenêtre **Logcat** affiche en temps réel les messages émis par l'appareil et par votre application (logs, exceptions, plantages). C'est l'outil n°1 pour diagnostiquer un problème à l'exécution.

Niveau	Usage
Verbose / Debug	Détails de mise au point

Niveau	Usage
Info	Événements normaux notables
Warn	Situations anormales non bloquantes
Error	Erreurs, exceptions, plantages (à chercher en priorité)

```
import android.util.Log

private const val TAG = "Calculatrice"

Log.d(TAG, "Expression évaluée : $expression")    // debug
Log.e(TAG, "Erreur de calcul", exception)        // erreur + pile d'appels
```

Lire une exception (stack trace)

Quand l'app plante, Logcat affiche une **pile d'appels** en rouge. La première ligne donne le type d'erreur (ex. `NullPointerException`) ; cherchez ensuite la première ligne qui mentionne **votre** package : c'est l'endroit à corriger. Filtrez Logcat par le nom de votre package ou par niveau `Error`.

AS.9 Inspecter l'interface : aperçu Compose, Live Edit & Layout Inspector

Avec Jetpack Compose, l'IDE offre des outils visuels puissants :

- **Aperçu @Preview** : affiche un composable sans lancer l'app ; mode `Interactive` pour cliquer, mode `Split` pour voir code et rendu côte à côte.
- **Live Edit** : modifie l'UI sur l'appareil/l'émulateur **sans relancer** l'application.
- **Layout Inspector** : inspecte l'arbre des composants d'une app en cours d'exécution, avec leurs propriétés et le **compteur de recompositions** (clé pour la performance Compose).

Multipreview

Annotez vos composables avec plusieurs `@Preview` (clair/sombre, petites/grandes polices, différentes tailles d'écran) pour vérifier d'un coup d'œil que l'UI s'adapte. Très utile pour l'UX et l'accessibilité.

AS.10 Le Profiler : mesurer les performances

Le **Profiler** (App Inspection / Profiler) mesure en temps réel la consommation de votre application. Indispensable dès que l'app rame, chauffe ou vide la batterie.

Profil	Ce qu'il mesure
CPU	Charge processeur, traces des threads, méthodes coûteuses
Memory	Mémoire utilisée, allocations, fuites mémoire
Energy	Consommation d'énergie (CPU, réseau, GPS)
Network	Requêtes réseau, taille et durée des échanges

Mesurer avant d'optimiser

N'optimisez jamais « au hasard ». Reproduisez le ralentissement pendant un enregistrement du Profiler, identifiez la cause (méthode lente, trop d'allocations, requête réseau bloquante), corrigez, puis re-mesurez.

AS.11 App Inspection : base de données, réseau, tâches

La fenêtre **App Inspection** regroupe des inspecteurs qui examinent une application en cours d'exécution :

- **Database Inspector** : explore en direct la base de données Room/SQLite, lance des requêtes, modifie des valeurs.
- **Network Inspector** : visualise les requêtes réseau (Retrofit/OkHttp), en-têtes et réponses.
- **Background Task Inspector** : suit les tâches WorkManager planifiées.

AS.12 APK Analyzer et Build Analyzer

L'**APK Analyzer** ([Build](#) → [Analyze APK...](#)) ouvre un APK et montre ce qu'il contient : taille de chaque composant (code dex, ressources, bibliothèques natives). Utile pour comprendre et réduire la taille de l'application.

Le **Build Analyzer** (onglet après un build) identifie les tâches Gradle les plus lentes et propose des pistes pour accélérer la compilation.

AS.13 Le contrôle de version (Git) intégré

Android Studio intègre **Git**, indispensable pour le travail en équipe. On peut tout faire depuis l'IDE, sans ligne de commande.

Action	Où / comment
Initialiser / cloner	Menu Git (VCS) → Create Git Repository / Clone
Voir les changements	Fenêtre Commit (Ctrl/Cmd + K) : fichiers modifiés
Committer	Saisir un message clair, sélectionner les fichiers, Commit
Pousser / récupérer	Push (envoyer), Pull/Update (récupérer les changements de l'équipe)
Branches	Créer/changer de branche en bas à droite (barre d'état)
Résoudre un conflit	Outil de fusion à trois panneaux (Merge) lors d'un pull

Bonnes pratiques Git en équipe

Committez souvent avec des messages clairs ; travaillez sur des **branches** par fonctionnalité ; faites des **pull/merge** réguliers pour limiter les conflits ; ajoutez un `.gitignore` Android (pour exclure `build/`, fichiers locaux, etc.). Hébergez le dépôt sur GitHub/GitLab.

AS.14 Raccourcis clavier essentiels

Quelques raccourcis font gagner un temps précieux (format Windows·Linux / macOS) :

Action	Windows / Linux	macOS
Rechercher partout	Maj Maj	⌘ ⌘
Complétion de code	Ctrl + Espace	^ Espace
Correction rapide / imports	Alt + Entrée	⌘ ↵
Renommer (refactor)	Shift + F6	⌘ F6
Reformater le code	Ctrl + Alt + L	⌘ ⌘ L
Aller à la déclaration	Ctrl + B	⌘ B
Rechercher les usages	Alt + F7	⌘ F7
Exécuter (Run)	Maj + F10	^ R
Déboguer (Debug)	Maj + F9	^ D
Commenter une ligne	Ctrl + /	⌘ /
Dupliquer une ligne	Ctrl + D	⌘ D
Générer (constructeur...)	Alt + Inser	⌘ N

Apprendre les raccourcis

Activez [Help](#) → [Productivity Guide](#) et la [Key Promoter](#) pour que l'IDE vous rappelle le raccourci d'une action que vous faites à la souris. Trois ou quatre raccourcis bien intégrés suffisent à transformer votre vitesse.

AS.15 Dépannage de l'IDE — réflexes utiles

Symptôme	Réflexe
Le code n'est pas reconnu / rouge partout	Gradle Sync ; sinon File → Invalidate Caches / Restart
Une dépendance reste introuvable	Vérifier les dépôts (google(), mavenCentral()) et la version
L'app ne s'installe pas sur l'appareil	Vérifier le débogage USB / l'autorisation, ou recréer l'AVD
Compilation très lente	Build Analyzer ; augmenter la mémoire dans gradle.properties
Émulateur ne démarre pas	Vérifier l'accélération matérielle et l'image système installée
Changement non pris en compte	« Sync Now » après modification d'un fichier Gradle

Le bon réflexe d'autonomie

Devant une erreur, lisez le message en entier, copiez la ligne clé dans un moteur de recherche, et consultez la documentation officielle (developer.android.com). 90 % des problèmes d'un débutant ont une réponse directe en ligne. Cette autonomie est l'un des objectifs du projet.

AS.16 Le terminal intégré et les outils en ligne de commande

Android Studio embarque un **terminal** (onglet en bas) ouvert à la racine du projet. On y lance les mêmes commandes que dans un terminal système, sans quitter l'IDE — pratique pour Gradle et Git.

```
# Exemples utiles depuis le terminal du projet
./gradlew assembleDebug      # compiler l'APK de debug
./gradlew test                # lancer les tests
git status                    # voir l'état du dépôt
git add . && git commit -m "feat: ajoute l'écran d'accueil"
adb devices                    # lister les appareils connectés
```

Interface graphique ou ligne de commande ?

Les deux sont équivalents : utilisez l'interface d'Android Studio (boutons, fenêtre Git) quand c'est plus simple, et le terminal pour des commandes précises ou répétitives. Savoir lire et lancer une commande Gradle est une compétence attendue.

AS.17 Personnaliser et étendre l'IDE

Quelques réglages améliorent le confort et la productivité :

- **Thème** : clair ou sombre ([Settings](#) → [Appearance](#)).
- **Police de l'éditeur** : taille et police adaptées (ligatures possibles).
- **Plugins** : [Settings](#) → [Plugins](#) pour ajouter des outils de productivité.
- **Live Templates** : raccourcis qui génèrent un bloc de code à partir d'un mot-clé.
- **Mémoire** : [Help](#) → [Change Memory Settings](#) sur les grosses configurations.

Ne pas se disperser

La personnalisation est utile mais secondaire : quelques réglages de confort suffisent. L'essentiel reste de maîtriser l'éditeur, le débogueur et Logcat. Évitez de passer plus de temps à configurer l'IDE qu'à coder.

Partie AND — Concepts fondamentaux d'Android

Jetpack Compose s'exécute au sein du système Android. Pour comprendre vraiment ce que fait votre application, il faut connaître quelques **concepts fondamentaux** de la plateforme : l'Activity et son cycle de vie, le Context, le manifeste, les permissions, les intents, les ressources et le thread principal. Ces notions reviennent en permanence et expliquent beaucoup de comportements (et de bugs).

AND.1 L'application et ses composants

Une application Android est un ensemble de **composants** déclarés dans le manifeste. Les quatre types historiques :

Composant	Rôle
Activity	Un écran avec lequel l'utilisateur interagit (point d'entrée de l'UI)
Service	Un traitement en arrière-plan, sans interface
BroadcastReceiver	Réagit à des événements système (batterie faible, démarrage...)
ContentProvider	Partage des données entre applications

En Compose moderne, l'essentiel se passe dans une seule **Activity** qui héberge toute l'UI. On utilise rarement les autres composants au début, mais il faut savoir qu'ils existent.

AND.2 L'Activity et son cycle de vie

Une **Activity** passe par une série d'états au fil de son existence : création, affichage, mise en pause, arrêt, destruction. Le système appelle des **méthodes de cycle de vie** à chaque transition.

Méthode	Moment
onCreate()	À la création : on installe l'UI (setContent { })
onStart()	L'activité devient visible
onResume()	L'activité passe au premier plan (interactive)
onPause()	Une autre activité passe devant (perte du focus)
onStop()	L'activité n'est plus visible
onDestroy()	L'activité est détruite (fermeture ou changement de configuration)

Pourquoi c'est crucial

Lors d'un **changement de configuration** (rotation, changement de langue), l'activité est **détruite et recréée**. C'est pourquoi un simple **remember** perd son état à la rotation, alors que le **ViewModel** — conçu pour survivre — le conserve. Comprendre le cycle de vie explique ce comportement.

Avec Compose, on n'écrit presque jamais ces méthodes soi-même : `setContent` et les effets (`LaunchedEffect`, `DisposableEffect`) gèrent le cycle de vie de l'UI. Mais savoir qu'il existe aide à comprendre la persistance de l'état.

AND.3 Le Context

Le `Context` est un objet central qui donne accès aux ressources et services du système : chaînes, thèmes, fichiers, services (notifications, capteurs)... L'`Activity` est elle-même un `Context`.

- En Compose, on l'obtient avec `LocalContext.current`.
- Il sert par exemple à lire une ressource, lancer une autre activité, ou accéder à `DataStore`.

Attention aux fuites mémoire

Ne conservez jamais une référence à long terme vers un `Context` d'activité (par ex. dans un objet statique) : cela empêche le système de libérer la mémoire. Préférez le `applicationContext` pour les usages durables.

AND.4 Le manifeste de l'application

Le fichier `AndroidManifest.xml` est la carte d'identité de l'application. Il déclare :

- Les **composants** (activités, services...) et le point d'entrée (l'activité avec le filtre `LAUNCHER`).
- Les **permissions** requises (Internet, caméra, localisation...).
- Les **métadonnées** : nom, icône, thème, version minimale.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.INTERNET" />

  <application
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/Theme.MonApp">

    <activity android:name=".MainActivity" android:exported="true">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>
</manifest>
```

AND.5 Les permissions

Pour accéder à des ressources sensibles (localisation, caméra, contacts), l'application doit **demandeur une permission**. On distingue :

- **Permissions normales** (ex. Internet) : accordées automatiquement, simplement déclarées dans le manifeste.
- **Permissions dangereuses** (ex. localisation, caméra) : il faut **les demander à l'utilisateur à l'exécution**, via une boîte de dialogue système.


```
// Demander une permission à l'exécution (Compose)
val lanceur = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestPermission()
) { accordee ->
    if (accordee) { /* utiliser la caméra */ }
}

Button(onClick = { lanceur.launch(Manifest.permission.CAMERA) }) {
    Text("Activer la caméra")
}
```

Demander au bon moment

Demandez une permission seulement quand elle est nécessaire, en expliquant pourquoi. Une demande de localisation dès l'ouverture, sans contexte, est souvent refusée. Respecter la vie privée fait partie de la qualité d'une application.

AND.6 Les intents

Un **Intent** est un message pour déclencher une action : ouvrir un autre écran, partager un texte, composer un numéro, ouvrir une page web. On distingue les intents **explicites** (cible précise) et **implicites** (le système choisit l'app capable de répondre).

```
val context = LocalContext.current

// Intent implicite : ouvrir une page web
val intent = Intent(Intent.ACTION_VIEW, Uri.parse("https://developer.android.com"))
context.startActivity(intent)

// Intent implicite : partager un texte
val partage = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Découvrez notre application !")
}
context.startActivity(Intent.createChooser(partage, "Partager via"))
```

AND.7 Les ressources et leurs qualificateurs

Les **ressources** (textes, couleurs, images, dimensions) vivent dans **res/**, séparées du code. Cela permet de les réutiliser et surtout de les **adapter** au contexte grâce à des **qualificateurs** de dossier.

Dossier	Contenu / adaptation
res/values/strings.xml	Textes (par défaut)
res/values-en/strings.xml	Textes en anglais (langue du téléphone)
res/values-night/	Couleurs/thème pour le mode sombre
res/drawable/	Images vectorielles et bitmap
res/mipmap-*/	Icônes de lanceur (plusieurs densités)

Externaliser les textes

N'écrivez jamais le texte affiché « en dur » dans le code Kotlin de production : placez-le dans `strings.xml` et référencez-le (`stringResource(R.string.titre)`). C'est indispensable pour traduire l'application et le signe d'un code professionnel.

AND.8 Le thread principal et la réactivité

Android dessine l'interface sur un unique **thread principal** (UI thread). Si vous y exécutez une tâche longue (réseau, base de données), l'interface **se fige** — et au-delà de quelques secondes, le système affiche « l'application ne répond pas » (ANR).

La règle d'or

Ne bloquez jamais le thread principal. Les **coroutines** (Partie 1 et 5) déplacent les tâches longues sur d'autres threads (`Dispatchers.IO`) tout en gardant un code lisible. C'est la raison d'être des fonctions `suspend`.

Avec Compose, la recomposition s'effectue aussi sur le thread principal : vos composables doivent rester **rapides** et déléguer tout travail lourd au ViewModel et aux coroutines.

Partie 1 — Kotlin essentiel pour Compose

Compose s'appuie intensément sur certaines fonctionnalités de Kotlin. Ce chapitre condense le minimum vital. Si vous connaissez déjà Kotlin, parcourez-le rapidement.

1.1 Variables, types et nullabilité

```
val nom = "Alice"      // immuable (préféré) – équivaut à final
var compteur = 0       // mutable
compteur += 1

val age: Int = 30      // type explicite
val prix = 9.99        // Double inféré

// Nullabilité : un type ne peut être null que s'il finit par ?
var texte: String? = null
val longueur = texte?.length ?: 0 // safe-call + Elvis : 0 si null
```

- **val** crée une référence non réassignable — à privilégier partout où c'est possible.
- Le **système de types nullable (?)** élimine la majorité des `NullPointerException` à la compilation.

1.2 Fonctions et lambdas

```
// Fonction simple
fun somme(a: Int, b: Int): Int = a + b

// Paramètres nommés et valeurs par défaut
fun saluer(nom: String, poli: Boolean = true) { /* ... */ }
saluer(nom = "Bob")

// Lambda : une fonction anonyme passée en argument
val doubler: (Int) -> Int = { x -> x * 2 }

// Trailing lambda : si le dernier paramètre est une lambda,
// on la sort des parenthèses – c'est LA syntaxe de Compose.
Button(onClick = { /* clic */ }) {
    Text("Valider")
}
```

Pourquoi c'est crucial pour Compose

La **trailing lambda** explique la syntaxe que vous verrez partout : `Column { ... }, Card { ... }`. Le bloc entre accolades est en réalité le **contenu** (un paramètre de type fonction) du composable.

1.3 Classes, data class et objets

```
// data class : porte des données, génère equals/hashCode/copy/toString
data class Utilisateur(val id: Int, val nom: String, val actif: Boolean = true)

val u = Utilisateur(1, "Alice")
val u2 = u.copy(actif = false) // copie en modifiant un champ

// enum
```

```
enum class Statut { CHARGEMENT, SUCCES, ERREUR }

// sealed : hiérarchie fermée, parfaite pour modéliser un état d'écran
sealed interface UiState {
    object Loading : UiState
    data class Success(val data: List<Utilisateur>) : UiState
    data class Error(val message: String) : UiState
}
```

Les **data class** et **sealed interface** sont les outils de prédilection pour modéliser l'état d'une interface — on y reviendra en Partie 3.

1.4 Coroutines : l'asynchrone simplifié

Les **coroutines** permettent d'écrire du code asynchrone (réseau, base de données) de façon séquentielle, sans bloquer le thread principal. Une fonction **suspend** peut être mise en pause puis reprise.

```
suspend fun chargerUtilisateur(id: Int): Utilisateur {
    delay(1000) // suspension non bloquante
    return Utilisateur(id, "Alice")
}

// On lance une coroutine dans un scope
viewModelScope.launch {
    val u = chargerUtilisateur(1) // attend sans bloquer l'UI
    // mise à jour de l'état...
}
```

Notez bien

Compose et les coroutines sont étroitement liés : les effets de bord (LaunchedEffect) et les flux de données (Flow) reposent sur les coroutines. Une bonne compréhension ici facilite toute la suite.

1.5 Collections et programmation fonctionnelle

Kotlin offre une API de collections riche et expressive. En Compose, on manipule constamment des listes pour produire de l'UI ; maîtriser ces opérateurs est indispensable.

```
val nombres = listOf(1, 2, 3, 4, 5, 6)

val pairs = nombres.filter { it % 2 == 0 } // [2, 4, 6]
val doubles = nombres.map { it * 2 } // [2, 4, 6, 8, 10, 12]
val somme = nombres.sumOf { it } // 21
val premierGrand = nombres.firstOrNull { it > 4 } // 5
val groupes = nombres.groupBy { it % 2 == 0 } // {false=[1,3,5], true=[2,4,6]}

// Chaînage : noms des utilisateurs actifs, triés
val noms = utilisateurs
    .filter { it.actif }
    .sortedBy { it.nom }
    .map { it.nom }
```

it désigne l'argument implicite d'une lambda à un seul paramètre. Ces opérateurs renvoient de nouvelles collections (immuables), ce qui s'accorde parfaitement avec le modèle d'état de Compose.

1.6 Les fonctions de portée (scope functions)

Cinq fonctions — `let`, `run`, `with`, `apply`, `also` — permettent d'exécuter un bloc dans le contexte d'un objet. Elles rendent le code plus concis.

Fonction	Objet référencé	Renvoie	Usage typique
<code>let</code>	<code>it</code>	résultat du bloc	Exécuter sur une valeur non-nulle (<code>?.let</code>)
<code>apply</code>	<code>this</code>	l'objet lui-même	Configurer un objet (builder)
<code>also</code>	<code>it</code>	l'objet lui-même	Effet secondaire (log) sans rompre la chaîne
<code>run / with</code>	<code>this</code>	résultat du bloc	Calculer un résultat à partir d'un objet

```
// ?.let : n'exécute le bloc que si la valeur n'est pas null
utilisateur?.let { afficherProfil(it) }

// apply : configure et renvoie l'objet
val intent = Intent(this, DetailActivity::class.java).apply {
    putExtra("id", userId)
    flags = Intent.FLAG_ACTIVITY_NEW_TASK
}
```

1.7 Interfaces, héritage et génériques

Kotlin permet l'abstraction par **interfaces** et **classes abstraites**, et la réutilisation de code par les **génériques**.

```
// Interface : un contrat
interface Repository {
    suspend fun charger(): List<Utilisateur>
}

// Implémentation
class RepositoryReseau(private val api: ApiService) : Repository {
    override suspend fun charger() = api.getUtilisateurs()
}

// Classe abstraite : partage du code + méthodes à implémenter
abstract class Ecran(val titre: String) {
    abstract fun afficher()
}

// Générique : une boîte qui marche pour n'importe quel type T
class Boite<T>(val contenu: T) {
    fun obtenir(): T = contenu
}
val b = Boite("Bonjour") // Boite<String>
```

Les interfaces sont au cœur de l'architecture testable : on dépend d'une interface (`Repository`) et on injecte une implémentation réelle en production, ou un faux en test.

1.8 La gestion des exceptions

Une **exception** signale une erreur à l'exécution. On l'attrape avec `try / catch` pour réagir proprement plutôt que de planter.

```
fun diviser(a: Int, b: Int): Int {
    return try {
        a / b
    } catch (e: ArithmeticException) {
        println("Erreur : division par zéro")
        0
    } finally {
        println("Calcul terminé") // toujours exécuté
    }
}

// Lever volontairement une exception
fun retirer(montant: Int) {
    if (montant <= 0) throw IllegalArgumentException("Montant invalide")
}
```

Échouer proprement

Anticipez les cas d'erreur (réseau coupé, saisie invalide, division par zéro) et transformez-les en messages clairs pour l'utilisateur. Une application qui ne plante jamais inspire confiance — c'est un critère de qualité du projet.

1.9 Fonctions d'ordre supérieur & séquences

Une **fonction d'ordre supérieur** prend une fonction en paramètre ou en renvoie une. C'est ce qui rend Kotlin (et Compose) si expressif.

```
// Prend une opération en paramètre
fun appliquer(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
    return operation(a, b)
}

val somme = appliquer(3, 4) { x, y -> x + y } // 7
val produit = appliquer(3, 4) { x, y -> x * y } // 12

// Séquences : évaluation paresseuse pour de gros volumes
val resultat = (1..1_000_000).asSequence()
    .map { it * 2 }
    .filter { it % 3 == 0 }
    .first() // ne calcule que le strict nécessaire
```

Les **Sequence** évaluent les opérations « à la demande » : utiles pour de longues chaînes de transformations sur de gros volumes, car elles évitent de créer des listes intermédiaires.

1.10 Structures de contrôle expressives

En Kotlin, `if` et `when` sont des **expressions** : ils renvoient une valeur. Cela rend le code concis et sûr.

```
// if comme expression
val signe = if (x >= 0) "positif" else "négatif"

// when : remplace les longues cascades de if/else
```

```
val categorie = when {  
    age < 13 -> "enfant"  
    age < 18 -> "adolescent"  
    else      -> "adulte"  
}  
  
// when sur une valeur, exhaustif sur une sealed/enum  
val message = when (etat) {  
    UiState.Loading      -> "Chargement..."  
    is UiState.Success   -> "${etat.data.size} éléments"  
    is UiState.Error     -> etat.message  
}  
  
// Smart cast : après un test de type, la variable est automatiquement convertie  
fun longueur(x: Any): Int = if (x is String) x.length else 0
```

Le when exhaustif, allié de la sûreté

Sur une `sealed interface` ou une `enum`, le compilateur exige que le `when` couvre tous les cas (pas de `else` oublié). Si vous ajoutez un nouvel état plus tard, le code ne compilera plus tant que vous ne l'aurez pas traité : les bugs sont évités dès la compilation.

Partie 2 — Les fondamentaux de Compose

2.1 Anatomie d'un composable

Un **composable** est une fonction Kotlin annotée `@Composable` qui décrit une portion d'interface. Par convention son nom commence par une majuscule et il ne retourne rien (il « émet » de l'UI).

```
@Composable
fun Carte(nom: String, sousTitre: String) {
    Card(modifier = Modifier.padding(8.dp)) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(text = nom, style = MaterialTheme.typography.titleLarge)
            Text(text = sousTitre, style = MaterialTheme.typography.bodyMedium)
        }
    }
}
```

- **Composabilité** : un composable peut en appeler d'autres, formant un arbre.
- **Pas de valeur de retour** : un composable décrit l'UI par effet, il n'en renvoie pas.
- **Idempotence** : pour un même état, il doit produire le même rendu (pas de logique imprévisible à l'intérieur).

2.2 Les layouts fondamentaux : Column, Row, Box

Trois conteneurs structurent 90 % des écrans :

Composable	Disposition	Alignement principal
Column	Empile verticalement	verticalArrangement / horizontalAlignment
Row	Aligne horizontalement	horizontalArrangement / verticalAlignment
Box	Superpose (axe z)	contentAlignment

```
Column(
    modifier = Modifier.fillMaxSize().padding(16.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp),
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Titre")
    Row(horizontalArrangement = Arrangement.SpaceBetween,
        modifier = Modifier.fillMaxWidth()) {
        Text("Gauche")
        Text("Droite")
    }
    Box(contentAlignment = Alignment.Center,
        modifier = Modifier.size(100.dp).background(Color.LightGray)) {
        Text("Centré")
    }
}
```


2.3 Les modifieurs

Un **Modifieur** décore ou augmente un composable : taille, marge, fond, clic, bordure... Les modifieurs se **chaînent**, et **l'ordre compte**.

```
Text(
    text = "Cliquez-moi",
    modifier = Modifier
        .padding(8.dp)           // marge AUTOUR
        .background(Color.Yellow) // fond
        .padding(16.dp)         // marge INTÉRIEURE (après le fond)
        .clickable { /* action */ }
)
```

Piège classique : l'ordre des modifieurs

Dans l'exemple ci-dessus, le premier **padding** s'applique avant le fond (espace externe transparent), tandis que le second s'applique après (espace coloré). Inverser l'ordre change complètement le rendu. Règle : on lit un modifieur de **l'extérieur vers l'intérieur**.

2.4 Composants Material 3 courants

Material 3 fournit une riche bibliothèque de composants prêts à l'emploi :

- **Text**, **Button**, **OutlinedButton**, **TextButton**, **Icon**, **Image**
- **TextField** et **OutlinedTextField** pour la saisie
- **Card**, **Surface**, **HorizontalDivider**, **Switch**, **Checkbox**, **Slider**
- **Scaffold** : la structure d'écran (barre du haut, barre du bas, FAB, contenu)

```
@Composable
fun EcranAccueil() {
    Scaffold(
        topBar = { TopAppBar(title = { Text("Mon application") }) },
        floatingActionButton = {
            FloatingActionButton(onClick = { /* ... */ }) {
                Icon(Icons.Default.Add, contentDescription = "Ajouter")
            }
        }
    ) { innerPadding ->
        Column(modifier = Modifier.padding(innerPadding)) {
            Text("Contenu principal")
        }
    }
}
```

Toujours appliquer innerPadding

Le **Scaffold** fournit un **innerPadding** qui tient compte des barres. **Vous devez l'appliquer** au contenu (via **Modifier.padding(innerPadding)**), sinon votre UI passera sous la barre du haut.

2.5 L'état : remember et mutableStateOf

Le cœur de Compose est la gestion de **l'état**. Un état est une donnée qui, lorsqu'elle change, déclenche la **recomposition** — la ré-exécution des composables qui la lisent, pour mettre à jour l'écran.

```
@Composable
fun Compteur() {
    // mutableStateOf crée un état observable
    // remember le conserve entre les recompositions
    var compte by remember { mutableStateOf(0) }

    Button(onClick = { compte++ }) {
        Text("Cliqué $compte fois")
    }
}
```

Trois éléments à distinguer :

- `mutableStateOf(0)` — crée un conteneur d'état observable.
- `remember { ... }` — mémorise la valeur d'une recomposition à l'autre (sinon elle serait recréée à chaque fois).
- `by` — délégation de propriété : permet d'écrire `compte` au lieu de `compte.value`.

remember ne survit pas à tout

Un état `remember` est perdu lors d'un changement de configuration (rotation de l'écran). Pour le préserver, utilisez `rememberSaveable` : `var compte by rememberSaveable { mutableStateOf(0) }`.

2.6 Le state hoisting (remontée d'état)

Un composable qui détient son propre état est dit **stateful**. Pour le rendre réutilisable et testable, on « remonte » l'état vers l'appelant : le composable devient **stateless** et reçoit la valeur + un callback. C'est le **state hoisting**, un patron fondamental.

```
// Stateless : ne possède pas l'état, le reçoit
@Composable
fun CompteurAffichage(compte: Int, onIncrement: () -> Unit) {
    Button(onClick = onIncrement) {
        Text("Cliqué $compte fois")
    }
}

// Le parent détient l'état (source de vérité unique)
@Composable
fun Ecran() {
    var compte by rememberSaveable { mutableStateOf(0) }
    CompteurAffichage(compte = compte, onIncrement = { compte++ })
}
```

Le patron « state down, events up »

L'état descend par les paramètres, les événements remontent par les callbacks. C'est le flux de données unidirectionnel (UDF) que l'on généralisera avec le ViewModel en Partie 3. Un composable stateless est plus facile à prévisualiser, à réutiliser et à tester.

2.7 Afficher des listes : LazyColumn et LazyRow

Pour de longues listes, n'utilisez jamais une `Column` avec un `forEach` (tout serait composé d'un coup). Préférez `LazyColumn` / `LazyRow`, qui ne composent que les éléments visibles à l'écran.

```

@Composable
fun ListeUtilisateurs(utilisateurs: List<Utilisateur>) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        items(
            items = utilisateurs,
            key = { it.id } // clé stable : performances + animations
        ) { user ->
            Carte(nom = user.nom, sousTitre = "ID ${user.id}")
        }
    }
}

```

Toujours fournir une key

La **key** identifie chaque élément de façon stable. Sans elle, Compose se base sur la position, ce qui provoque des recompositions inutiles et casse les animations lors d'insertions/suppressions.

2.8 Les effets de bord (side effects)

Un composable doit rester « pur ». Pour exécuter du code qui interagit avec l'extérieur (lancer une requête, démarrer un timer, afficher un snackbar) au bon moment du cycle de vie, on utilise des **effets** :

Effet	Usage typique
LaunchedEffect(key)	Lancer une coroutine quand le composable entre / la clé change
rememberCoroutineScope()	Obtenir un scope pour lancer une coroutine sur un événement (clic)
DisposableEffect(key)	Enregistrer puis libérer une ressource (listener, callback)
derivedStateOf { }	Calculer un état dérivé sans recomposer à chaque changement
rememberUpdatedState()	Capturer la dernière valeur dans un effet de longue durée
snapshotFlow { }	Convertir un State Compose en Flow

```

@Composable
fun EcranProfil(userId: Int, viewModel: ProfilViewModel) {
    // Relancé uniquement si userId change
    LaunchedEffect(userId) {
        viewModel.charger(userId)
    }

    val scope = rememberCoroutineScope()
    Button(onClick = {
        scope.launch { viewModel.rafraichir() } // coroutine sur clic
    }) { Text("Rafraîchir") }
}

```

La règle d'or des effets

Tout ce qui « sort » du monde Compose (réseau, BD, logs, navigation, snackbar) doit passer par un effet. N'appellez **jamais** une fonction suspend ou un effet de bord directement dans le corps d'un composable : il est ré-exécuté à chaque recomposition.

2.9 La saisie de texte et la validation

Un champ de saisie en Compose est **stateless** : il ne stocke pas son contenu, vous lui fournissez la **value** et un callback **onValueChange**. C'est le state hoisting appliqué à la saisie.

```
@Composable
fun ChampEmail() {
    var email by rememberSaveable { mutableStateOf("") }
    val estValide = remember(email) {
        android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches()
    }

    OutlinedTextField(
        value = email,
        onValueChange = { email = it },
        label = { Text("E-mail") },
        singleLine = true,
        isError = email.isNotEmpty() && !estValide,
        supportingText = {
            if (email.isNotEmpty() && !estValide) Text("Adresse invalide")
        },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email)
    )
}
```

Validation et état dérivé

La validité est un **état dérivé** du texte : on la recalcule avec **remember(email)** plutôt que de la stocker séparément, ce qui évite toute désynchronisation. Pour des formulaires complexes, remontez l'état dans le ViewModel.

2.10 Gérer les gestes et le défilement

Au-delà de **clickable**, Compose expose des modifieurs pour le défilement et des gestes riches via **pointerInput**.

```
// Rendre une Column défilante
Column(modifier = Modifier.verticalScroll(rememberScrollState())) {
    repeat(50) { Text("Ligne $it") }
}

// Gestes personnalisés : glisser et taper deux fois
Box(modifier = Modifier
    .pointerInput(Unit) {
        detectTapGestures(onDoubleTap = { /* zoom */ })
    }
    .pointerInput(Unit) {
        detectDragGestures { change, drag -> /* déplacer */ }
    }
)
```

- Pour une liste, le défilement est intégré à `LazyColumn` — n'ajoutez pas `verticalScroll` dessus (conflit).
- Combinez `Modifier.draggable`, `swipeable` et `Animatable` pour des interactions avancées (swipe-to-dismiss).

2.11 Grilles et en-têtes collants

Pour des dispositions en grille, utilisez `LazyVerticalGrid` ; pour des sections avec titres fixes, `stickyHeader`.

```
LazyVerticalGrid(
    columns = GridCells.Fixed(2),          // ou GridCells.Adaptive(120.dp)
    contentPadding = PaddingValues(12.dp),
    horizontalArrangement = Arrangement.spacedBy(8.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    items(photos, key = { it.id }) { photo -> VignettePhoto(photo) }
}

LazyColumn {
    contacts.groupBy { it.nom.first() }.forEach { (lettre, groupe) ->
        stickyHeader { EnTeteSection(lettre.toString()) }
        items(groupe, key = { it.id }) { ContactRow(it) }
    }
}
```

2.12 Partager des valeurs : CompositionLocal

Un `CompositionLocal` diffuse implicitement une valeur dans tout un sous-arbre, sans la passer en paramètre à chaque composable. C'est le mécanisme derrière `MaterialTheme` ou `LocalContext`.

```
// Quelques CompositionLocal fournis par le framework
val context = LocalContext.current
val configuration = LocalConfiguration.current
val density = LocalDensity.current

// En définir un (à réserver aux dépendances transverses)
val LocalUtilisateur = compositionLocalOf<Utilisateur?> { null }

CompositionLocalProvider(LocalUtilisateur provides utilisateurCourant) {
    EcranProfil() // peut lire LocalUtilisateur.current en profondeur
}
```

À utiliser avec parcimonie

Le `CompositionLocal` rend les dépendances implicites : pratique pour des éléments réellement transverses (thème, densité, locale), mais à éviter pour les données métier, qui doivent rester explicites via les paramètres. Trop de `CompositionLocal` nuit à la lisibilité et à la testabilité.

2.13 Boîtes de dialogue et feuilles modales

`Compose` fournit des composants prêts pour les surfaces temporaires : dialogues, menus et bottom sheets.

```

var afficherDialog by remember { mutableStateOf(false) }

if (afficherDialog) {
    AlertDialog(
        onDismissRequest = { afficherDialog = false },
        title = { Text("Supprimer ?") },
        text = { Text("Cette action est irréversible.") },
        confirmButton = {
            TextButton(onClick = { supprimer(); afficherDialog = false }) {
                Text("Supprimer")
            }
        },
        dismissButton = {
            TextButton(onClick = { afficherDialog = false }) { Text("Annuler") }
        }
    )
}

// Snackbar via le Scaffold
val snackbarHostState = remember { SnackbarHostState() }
val scope = rememberCoroutineScope()
scope.launch { snackbarHostState.showSnackbar("Enregistré") }

```

2.14 Galerie de composants Material 3

Au-delà des composants de base, Material 3 offre un large catalogue. En voici les plus utiles pour vos projets :

Composant	Usage
Card / ElevatedCard	Regrouper du contenu lié dans une surface surélevée
Chip / FilterChip	Étiquettes sélectionnables (filtres, choix)
Switch / Checkbox / RadioButton	Choix binaires et exclusifs
Slider	Sélection d'une valeur dans une plage
LinearProgressIndicator	Progression déterminée ou indéterminée
TabRow / Tab	Onglets de navigation interne
NavigationBar	Barre de navigation principale (bas d'écran)
ModalBottomSheet	Feuille modale glissant depuis le bas
AssistChip / Badge	Actions contextuelles et pastilles de notification

```

// Exemples concis
FilterChip(selected = actif, onClick = { actif = !actif },
    label = { Text("Disponible") })

Slider(value = volume, onValueChange = { volume = it }, valueRange = 0f..100f)

var afficherSheet by remember { mutableStateOf(false) }
if (afficherSheet) {
    ModalBottomSheet(onDismissRequest = { afficherSheet = false }) {

```

```

        Text("Contenu de la feuille", Modifier.padding(16.dp))
    }
}

```

Explorer le catalogue

L'application officielle « Material 3 Catalog » et la documentation montrent chaque composant en action. Réutiliser les composants standard garantit une UI cohérente, accessible et conforme aux attentes des utilisateurs.

2.15 Les trois phases de Compose

À chaque image (frame), Compose exécute trois phases. Les comprendre éclaire la performance et le placement des éléments.

Phase	Question	Exemple d'API
Composition	Quoi afficher ?	Fonctions @Composable, état lu
Layout	Où et quelle taille ?	Mesure et placement, Modifier.offset { }
Drawing	Comment dessiner ?	Canvas, drawBehind, graphicsLayer

Différer la lecture de l'état

Si un état ne sert qu'au placement ou au dessin (ex. un décalage animé), lisez-le dans une lambda (`Modifier.offset { ... }`) plutôt que directement : Compose saute alors la phase de Composition et ne refait que le Layout/Drawing — un gain de performance notable.

2.16 Agencements avancés : poids, ConstraintLayout, FlowRow

Plusieurs outils complètent Column/Row/Box pour des mises en page riches :

- `Modifier.weight(f)` : répartit l'espace restant d'une Row/Column proportionnellement (ex. deux colonnes 1/3 – 2/3).
- `ConstraintLayout` : positionne les éléments les uns par rapport aux autres (utile pour des écrans complexes, comme l'ancien XML).
- `FlowRow` / `FlowColumn` : disposent les éléments en les renvoyant à la ligne quand l'espace manque (ex. nuage d'étiquettes).

```

// weight : la colonne de gauche prend 1 part, celle de droite 2 parts
Row(Modifier.fillMaxWidth()) {
    Text("Libellé", Modifier.weight(1f))
    Text("Valeur", Modifier.weight(2f))
}

// FlowRow : des chips qui passent à la ligne automatiquement
FlowRow(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
    etiquettes.forEach { AssistChip(onClick = {}, label = { Text(it) }) }
}

```

2.17 Exercices — Partie 2

1. Créez un composable Profil affichant un avatar (Icon), un nom et un bouton « Suivre » qui bascule en « Suivi » au clic (état local).
2. Transformez ce composable en version stateless + parent stateful (state hoisting).
3. Affichez une LazyColumn de 50 éléments numérotés, chacun dans une Card avec une key stable.
4. Ajoutez un Scaffold avec une TopAppBar et un FloatingActionButton qui ajoute un élément à la liste.
5. Bonus : utilisez rememberSaveable pour que le compteur de « likes » survive à la rotation.

Partie 3 — Architecture & navigation

3.1 L'architecture recommandée

Google recommande une architecture en couches reposant sur le **flux de données unidirectionnel (UDF)** et la **séparation des responsabilités**. Trois couches :

Couche	Responsabilité	Composants typiques
UI	Afficher l'état, capter les événements utilisateur	Composables, ViewModel, UiState
Domaine (optionnel)	Logique métier réutilisable	Use cases / Interactors
Données	Fournir et persister les données	Repository, sources réseau/locale

Principe directeur

L'UI ne contient aucune logique métier : elle observe un état et émet des événements. Le ViewModel détient l'état et orchestre la logique. Le Repository est l'unique source de vérité des données. Cette séparation rend l'app testable et évolutive.

3.2 Le ViewModel et l'état d'écran

Le [ViewModel](#) survit aux changements de configuration et détient l'état de l'écran, exposé sous forme de [StateFlow](#). On modélise souvent l'état avec une [data class](#) ou une [sealed interface](#).

```
data class AccueilUiState(
    val isLoading: Boolean = false,
    val utilisateurs: List<Utilisateur> = emptyList(),
    val erreur: String? = null
)

class AccueilViewModel(
    private val repository: UtilisateurRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow(AccueilUiState())
    val uiState: StateFlow<AccueilUiState> = _uiState.asStateFlow()

    init { charger() }

    fun charger() {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, erreur = null) }
            try {
                val data = repository.getUtilisateurs()
                _uiState.update { it.copy(isLoading = false, utilisateurs = data) }
            } catch (e: Exception) {
                _uiState.update { it.copy(isLoading = false, erreur = e.message) }
            }
        }
    }
}
```

```
}
}
```

3.3 Connecter le ViewModel à l'UI

Côté composable, on récupère le ViewModel avec `viewModel()` et on collecte l'état avec `collectAsStateWithLifecycle()`, qui suspend la collecte quand l'écran n'est plus visible.

```
@Composable
fun EcranAccueil(viewModel: AccueilViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()

    when {
        uiState.isLoading -> CircularProgressIndicator()
        uiState.erreur != null -> Text("Erreur : ${uiState.erreur}")
        else -> ListeUtilisateurs(uiState.utilisateurs)
    }
}
```

collectAsStateWithLifecycle

Préférez-la à `collectAsState()` : elle respecte le cycle de vie et évite de continuer à collecter en arrière-plan (économie de batterie et de ressources). Elle provient de [lifecycle-runtime-compose](#).

3.4 La navigation avec Navigation Compose

La navigation entre écrans repose sur un `NavController` et un `NavHost` qui associe des **routes** à des composables. Les versions récentes proposent une navigation **type-safe** basée sur des objets sérialisables plutôt que des chaînes.

```
// Routes type-safe (avec kotlinx.serialization)
@Serializable object Accueil
@Serializable data class Detail(val userId: Int)

@Composable
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(navController, startDestination = Accueil) {
        composable<Accueil> {
            EcranAccueil(onUserClick = { id ->
                navController.navigate(Detail(userId = id))
            })
        }
        composable<Detail> { backStackEntry ->
            val detail: Detail = backStackEntry.toRoute()
            EcranDetail(userId = detail.userId,
                onBack = { navController.popBackStack() })
        }
    }
}
```

- `navigate(route)` empile une destination ; `popBackStack()` revient en arrière.
- La navigation type-safe évite les erreurs de chaînes et les conversions manuelles d'arguments.

3.5 L'injection de dépendances avec Hilt

Pour fournir automatiquement Repository, sources de données et ViewModels, on utilise [Hilt](#), la solution d'injection recommandée, bâtie sur Dagger.

```
@HiltAndroidApp
class MonApplication : Application()

@Module
@InstallIn(SingletonComponent::class)
object DataModule {
    @Provides @Singleton
    fun fournirRepository(api: ApiService): UtilisateurRepository =
        UtilisateurRepositoryImpl(api)
}

@HiltViewModel
class AccueilViewModel @Inject constructor(
    private val repository: UtilisateurRepository
) : ViewModel() { /* ... */ }

// Dans le composable :
@Composable
fun EcranAccueil(viewModel: AccueilViewModel = hiltViewModel()) { /* ... */ }
```

Pourquoi l'injection de dépendances ?

Elle découple la création des objets de leur utilisation : le ViewModel ne sait pas comment construire son Repository, il le reçoit. Résultat : un code testable (on injecte un faux Repository en test) et modulaire.

3.6 Les événements ponctuels (one-shot)

Le [StateFlow](#) décrit un état persistant. Mais certains événements ne doivent se produire **qu'une fois** (afficher un message, déclencher une navigation). Les exposer comme un état provoquerait leur ré-émission après une rotation. On utilise alors un [Channel](#) exposé en [Flow](#).

```
class FormulaireViewModel : ViewModel() {
    private val _evenements = Channel<Evenement>()
    val evenements = _evenements.receiveAsFlow()

    fun valider() = viewModelScope.launch {
        // ... logique ...
        _evenements.send(Evenement.NaviguerVersAccueil)
    }
}

sealed interface Evenement {
    data object NaviguerVersAccueil : Evenement
    data class Message(val texte: String) : Evenement
}

// Côté UI : on consomme l'événement une seule fois
LaunchedEffect(Unit) {
```

```
viewModel.evenements.collect { evt ->
    when (evt) {
        Evenement.NaviguerVersAccueil -> navController.navigate(Accueil)
        is Evenement.Message -> snackbarHostState.showSnackbar(evt.texte)
    }
}
```

État vs événement

Règle simple : si la donnée décrit « ce qui est affiché à l'écran maintenant », c'est un état (StateFlow). Si elle décrit « quelque chose qui vient de se produire », c'est un événement (Channel/SharedFlow). Confondre les deux est une source classique de bugs (messages répétés, navigations en boucle).

3.7 Préserver l'état avec SavedStateHandle

Le ViewModel survit aux rotations mais **pas** à la suppression du processus par le système (manque de mémoire). Pour préserver les données critiques, injectez un [SavedStateHandle](#) — il reçoit aussi automatiquement les arguments de navigation.

```
@HiltViewModel
class DetailViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val repository: UtilisateurRepository
) : ViewModel() {
    // Argument de route récupéré automatiquement
    private val route = savedStateHandle.toRoute<Detail>()

    // État de saisie qui survit à la mort du processus
    var brouillon by savedStateHandle.saveable { mutableStateOf("") }
    private set
}
```

3.8 Navigation à onglets et liens profonds

Pour une barre d'onglets, on combine une [NavigationBar](#) Material 3 et un [NavHost](#) imbriqué. Les **liens profonds** (deep links) permettent d'ouvrir un écran précis depuis une URL ou une notification.

```
Scaffold(
    bottomBar = {
        NavigationBar {
            destinations.forEach { dest ->
                NavigationBarItem(
                    selected = routeCourante == dest.route,
                    onClick = { navController.navigate(dest.route) },
                    icon = { Icon(dest.icone, contentDescription = dest.label) },
                    label = { Text(dest.label) }
                )
            }
        }
    }
) { padding ->
    NavHost(navController, startDestination = Accueil,
        modifier = Modifier.padding(padding)) {
```

```

        composable<Detail>{
            deepLinks = listOf(navDeepLink { uriPattern = "monapp://detail/{userId}" })
        } { /* ... */ }
    }
}

```

3.9 Le patron Repository en détail

Le **Repository** isole le reste de l'application des détails d'accès aux données. Le **ViewModel** ne sait pas si les données viennent du réseau, d'une base locale ou d'un cache : il demande simplement au **Repository**.

```

// Interface (contrat) – vit dans la couche données
interface MeteoRepository {
    fun observerMeteo(ville: String): Flow<Meteo>
    suspend fun rafraichir(ville: String)
}

// Implémentation : combine réseau + cache local
class MeteoRepositoryImpl(
    private val api: MeteoApi,
    private val dao: MeteoDao
) : MeteoRepository {
    override fun observerMeteo(ville: String) =
        dao.observer(ville).map { it.toModel() } // l'UI lit la base locale

    override suspend fun rafraichir(ville: String) {
        val distant = api.getMeteo(ville) // appel réseau
        dao.inserer(distant.toEntity()) // mise à jour de la base
    }
}

```

Avantages

Un **Repository** rend le code testable (on injecte un faux), interchangeable (changer de source sans toucher au **ViewModel**) et clair (chaque couche a une seule responsabilité).

3.10 Navigation imbriquée & portée du ViewModel

Pour des sections complexes (ex. un tunnel d'inscription en plusieurs écrans), on regroupe des destinations dans un **graphe imbriqué** avec `navigation(...)`. On peut alors **partager un ViewModel** entre ces écrans en le rattachant à l'entrée du graphe parent.

```

NavHost(navController, startDestination = "accueil") {
    composable("accueil") { /* ... */ }

    // Sous-graphe « inscription » avec ViewModel partagé
    navigation(startDestination = "etape1", route = "inscription") {
        composable("etape1") { entry ->
            val vm = entry.sharedViewModel<InscriptionViewModel>(navController)
            Etape1(vm)
        }
        composable("etape2") { entry ->
            val vm = entry.sharedViewModel<InscriptionViewModel>(navController)
            Etape2(vm)
        }
    }
}

```

```
}  
}  
}
```

Par défaut, chaque écran a son propre ViewModel, dont la durée de vie suit son entrée dans la pile de navigation. Le partage permet de conserver des données saisies tout au long d'un parcours.

3.11 Les portées avec Hilt

Hilt fournit des objets avec une **portée** (durée de vie) adaptée :

Portée	Durée de vie	Usage typique
@Singleton	Toute l'application	Repository, base de données, client réseau
@ViewModelScoped	Durée du ViewModel	Dépendances propres à un écran
@ActivityScoped	Durée de l'activité	Objets liés à l'écran hôte

Bien choisir la portée

Un Repository ou une base de données doit être @Singleton (un seul exemplaire partagé). Créer plusieurs bases ou clients réseau gaspille la mémoire et peut provoquer des incohérences.

3.12 Exercices — Partie 3

1. Créez un ViewModel exposant un `StateFlow<UiState>` avec les états Loading/Success/Error (sealed interface).
2. Branchez l'écran avec `collectAsStateWithLifecycle` et un `when` exhaustif sur l'état.
3. Mettez en place une navigation à deux écrans (liste → détail) en passant un identifiant type-safe.
4. Ajoutez Hilt : annotez l'Application, créez un module fournissant un Repository, injectez-le dans le ViewModel.

Partie 4 — UI avancée

4.1 Le thème Material 3

Material 3 (Material You) structure le thème autour de trois piliers : **color scheme**, **typography** et **shapes**. Le thème est un composable qui diffuse ces valeurs via [MaterialTheme](#).

```
private val LightColors = lightColorScheme(
    primary = Color(0xFF1B5E20),
    secondary = Color(0xFF558B2F),
    background = Color(0xFFFFD6D6)
)
private val DarkColors = darkColorScheme(primary = Color(0xFF81C784))

@Composable
fun MonAppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,           // couleurs dynamiques (Android 12+)
    content: @Composable () -> Unit
) {
    val colors = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val ctx = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(ctx) else dynamicLightColorScheme(ctx)
        }
        darkTheme -> DarkColors
        else -> LightColors
    }
    MaterialTheme(colorScheme = colors, typography = Typography, content = content)
}
```

- **Couleur dynamique** : sur Android 12+, l'app peut adopter la palette extraite du fond d'écran de l'utilisateur.
- **Thème sombre** : `isSystemInDarkTheme()` suit le réglage système ; fournissez toujours les deux palettes.
- Accédez aux valeurs via `MaterialTheme.colorScheme.primary`, `MaterialTheme.typography.titleLarge`, etc.

4.2 Les animations

Compose rend l'animation déclarative et fluide. Quelques outils, du plus simple au plus puissant :

API	Pour quoi faire
<code>animate*AsState</code>	Animer une seule valeur (couleur, taille, alpha) vers une cible
<code>AnimatedVisibility</code>	Apparition / disparition d'un élément
<code>Crossfade</code>	Transition en fondu entre deux contenus
<code>AnimatedContent</code>	Transition riche entre états (avec direction)
<code>updateTransition</code>	Coordonner plusieurs animations sur un même état
<code>Animatable</code>	Contrôle bas niveau, animations basées sur des gestes

```

@Composable
fun BoutonAnime(selectionne: Boolean) {
    val couleur by animateColorAsState(
        targetValue = if (selectionne) Color(0xFF2E7D32) else Color.LightGray,
        animationSpec = tween(durationMillis = 300),
        label = "couleurFond"
    )
    Box(modifier = Modifier.size(120.dp).background(couleur))

    AnimatedVisibility(visible = selectionne) {
        Text("Sélectionné !")
    }
}

```

Animer « gratuitement »

Beaucoup d'animations sont automatiques : ajoutez `Modifier.animateContentSize()` pour animer un changement de taille, ou `Modifier.animateItem()` dans une `LazyColumn` pour animer insertions, suppressions et déplacements.

4.3 Dessin et layouts personnalisés

Quand les composants standard ne suffisent pas, on dessine directement avec `Canvas` ou on crée un agencement sur mesure avec le composable `Layout`, qui donne accès aux phases **mesure** puis **placement**.

```

// Dessin custom
Canvas(modifier = Modifier.size(120.dp)) {
    drawCircle(color = Color(0xFF2E7D32), radius = size.minDimension / 3)
    drawLine(Color.White, start = Offset(0f, 0f),
        end = Offset(size.width, size.height), strokeWidth = 8f)
}

// Layout personnalisé : empile en diagonale
@Composable
fun ColonneDiagonale(content: @Composable () -> Unit) {
    Layout(content = content) { measurables, constraints ->
        val placeables = measurables.map { it.measure(constraints) }
        val hauteur = placeables.sumOf { it.height }
        layout(constraints.maxWidth, hauteur) {
            var y = 0
            placeables.forEachIndexed { i, p ->
                p.place(x = i * 20, y = y)
                y += p.height
            }
        }
    }
}

```

4.4 Comprendre et optimiser la performance

Compose travaille en trois phases : **Composition** (quoi afficher), **Layout** (mesure et placement) et **Drawing** (dessin). L'objectif de l'optimisation est de **minimiser les recompositions inutiles**.

Bonnes pratiques essentielles :

- **Stabilité** : préférez des types stables (data class immuables, collections immuables). Une `List<T>` mutable rend le composable instable et provoque des recompositions superflues — utilisez `kotlinx.collections.immutable`.
- **Différer la lecture de l'état** : lisez un état le plus tard possible (dans une lambda, ou via `Modifier.offset { ... }`) pour ne recomposer que la phase nécessaire.
- **Clés stables** dans les listes Lazy (voir 2.7).
- **derivedStateOf** pour les calculs dérivés coûteux qui ne doivent pas déclencher de recomposition à chaque frappe.
- **remember** les calculs coûteux et les objets créés dans la composition.
- **Baseline Profiles** : précompilez les chemins critiques pour accélérer le démarrage et le défilement.

Mesurer avant d'optimiser

Activez le **Layout Inspector** (compteur de recompositions) et générez les **rapports du compilateur Compose** pour repérer les composables instables. N'optimisez jamais à l'aveugle : mesurez, corrigez, re-mesurez.

4.5 Layouts adaptatifs (téléphones, tablettes, pliables)

Une app moderne doit s'adapter aux écrans variés. Les **Window Size Classes** classent la largeur disponible en trois catégories — `Compact`, `Medium`, `Expanded` — pour choisir une disposition.

```
@Composable
fun AppAdaptative(windowSizeClass: WindowSizeClass) {
    when (windowSizeClass.widthSizeClass) {
        WindowWidthSizeClass.Compact ->
            EcranListeDetailUneColonne()           // téléphone : liste OU détail
        else ->
            EcranListeDetailDeuxColonnes()          // tablette : liste ET détail
    }
}
```

- La bibliothèque `material3-adaptive` fournit des échafaudages prêts (ex. `ListDetailPaneScaffold`) pour les patrons liste/détail.
- Évitez de coder en dur des tailles ; raisonnez en termes de classes de taille et de `BoxWithConstraints` lorsque c'est nécessaire.

4.6 L'accessibilité

Une UI accessible est utilisable par tous, y compris avec un lecteur d'écran (TalkBack). Compose expose la **sémantique** de chaque élément, que vous pouvez enrichir.

```
Icon(
    imageVector = Icons.Default.Favorite,
    contentDescription = "Ajouter aux favoris" // lu par TalkBack ; null si purement
    décoratif
)

// Fusionner la sémantique d'un groupe et donner un libellé
Row(modifier = Modifier
```

```

.clickable { /* ... */ }
.semantics(mergeDescendants = true) {
    contentDescription = "Profil de Alice, 12 abonnés"
}
) { /* avatar + textes */ }

```

- Donnez un `contentDescription` à toute icône/image porteuse de sens ; mettez `null` pour le décoratif.
- Respectez une taille tactile minimale de 48 dp et un contraste de texte suffisant (ratio 4,5:1).
- Testez réellement avec TalkBack et l'outil [Accessibility Scanner](#).

4.7 Les animations pilotées par les gestes

Pour des interactions tangibles (glisser pour fermer, tirer pour rafraîchir), on combine [Animatable](#) et [pointerInput](#) afin que l'élément « suive le doigt » puis s'anime jusqu'à sa position finale.

```

@Composable
fun CarteGlissante(onDismiss: () -> Unit) {
    val offsetX = remember { Animatable(0f) }
    val scope = rememberCoroutineScope()

    Box(modifier = Modifier
        .offset { IntOffset(offsetX.value.roundToInt(), 0) }
        .pointerInput(Unit) {
            detectHorizontalDragGestures(
                onHorizontalDrag = { _, delta ->
                    scope.launch { offsetX.snapTo(offsetX.value + delta) }
                },
                onDragEnd = {
                    scope.launch {
                        if (abs(offsetX.value) > 300) onDismiss()
                        else offsetX.animateTo(0f) // revient en place
                    }
                }
            )
        }
    ) { /* contenu */ }
}

```

4.8 Typographie, formes et tokens de thème

Material 3 structure l'apparence autour de **tokens** : un ensemble cohérent de styles de texte, de formes et de couleurs accessibles partout via [MaterialTheme](#).

```

// Échelle typographique
Text("Titre", style = MaterialTheme.typography.headlineMedium)
Text("Corps", style = MaterialTheme.typography.bodyLarge)
Text("Légende", style = MaterialTheme.typography.labelSmall)

// Formes (coins arrondis cohérents)
Card(shape = MaterialTheme.shapes.large) { /* ... */ }

// Couleurs sémantiques (et non des valeurs codées en dur)
Surface(color = MaterialTheme.colorScheme.primaryContainer,
        contentColor = MaterialTheme.colorScheme.onPrimaryContainer) { /* ... */ }

```

Penser en rôles, pas en couleurs

Utilisez `colorScheme.primary`, `surface`, `error`... plutôt que `Color(0xFF...)`. Vos écrans s'adaptent alors automatiquement au mode sombre et à la couleur dynamique, sans retouche.

4.9 Afficher des images (ressources et réseau)

Pour les images locales, on utilise `painterResource`. Pour des images distantes (URL), la bibliothèque **Coil** est la référence en Compose.

```
// Image locale (ressource)
Image(painter = painterResource(R.drawable.logo), contentDescription = "Logo")

// Image distante avec Coil
AsyncImage(
    model = "https://exemple.com/photo.jpg",
    contentDescription = "Photo de profil",
    contentScale = ContentScale.Crop,
    modifier = Modifier.size(64.dp).clip(CircleShape)
)
```

- `contentScale` contrôle le cadrage (`Crop`, `Fit`, `FillBounds`).
- Donnez toujours un `contentDescription` (ou `null` si l'image est purement décorative) pour l'accessibilité.

4.10 Exercices — Partie 4

1. Définissez un thème clair/sombre personnalisé et basculez-le via un `Switch`.
2. Animez la couleur et la taille d'une carte lorsqu'on la sélectionne (`animateColorAsState` + `animateContentSize`).
3. Réalisez un `AnimatedContent` qui fait défiler un compteur avec une transition de glissement.
4. Dessinez une jauge circulaire de progression avec `Canvas`.
5. Bonus : créez un `Layout` personnalisé disposant ses enfants en grille fluide.

Partie 5 — Données & tests

5.1 Coroutines et Flow approfondis

Un **Flow** est un flux asynchrone de valeurs dans le temps (l'équivalent réactif d'une séquence). C'est la brique de base pour observer des données qui changent (BD locale, événements).

```
fun observerUtilisateurs(): Flow<List<Utilisateur>> = flow {
    while (true) {
        emit(api.getUtilisateurs())
        delay(30_000)           // rafraîchit toutes les 30 s
    }
}

// Opérateurs : map, filter, combine, debounce...
val nomsActifs: Flow<List<String>> =
    observerUtilisateurs()
        .map { liste -> liste.filter { it.actif }.map { it.nom } }

// Dans le ViewModel, on transforme un Flow froid en StateFlow chaud
val uiState: StateFlow<List<String>> = nomsActifs
    .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
```

- **StateFlow** : détient toujours une valeur courante, parfait pour l'état d'UI.
- **SharedFlow** : pour les événements ponctuels (navigation, message).

5.2 Le réseau avec Retrofit

Retrofit transforme une interface Kotlin en client HTTP. Combiné aux coroutines, les appels deviennent de simples fonctions **suspend**.

```
data class UtilisateurDto(val id: Int, val name: String)

interface ApiService {
    @GET("users")
    suspend fun getUtilisateurs(): List<UtilisateurDto>

    @GET("users/{id}")
    suspend fun getUtilisateur(@Path("id") id: Int): UtilisateurDto
}

val retrofit = Retrofit.Builder()
    .baseUrl("https://api.exemple.com/")
    .addConverterFactory(Json.asConverterFactory("application/json".toMediaType()))
    .build()

val api: ApiService = retrofit.create(ApiService::class.java)
```

5.3 La persistance locale avec Room

Room est une couche d'abstraction au-dessus de SQLite. On déclare une **entité**, un **DAO** et une **base**. Les requêtes peuvent renvoyer un **Flow** pour observer la base en temps réel.

```

@Entity(tableName = "utilisateurs")
data class UtilisateurEntity(
    @PrimaryKey val id: Int,
    val nom: String,
    val actif: Boolean
)

@Dao
interface UtilisateurDao {
    @Query("SELECT * FROM utilisateurs")
    fun observerTous(): Flow<List<UtilisateurEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun inserer(utilisateurs: List<UtilisateurEntity>)
}

@Database(entities = [UtilisateurEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun utilisateurDao(): UtilisateurDao
}

```

5.4 Assembler : le Repository

Le [Repository](#) combine sources réseau et locale et constitue la **source unique de vérité**. Patron courant : la base de données locale est la vérité, le réseau la rafraîchit (offline-first).

```

class UtilisateurRepositoryImpl(
    private val api: ApiService,
    private val dao: UtilisateurDao
) : UtilisateurRepository {

    // L'UI observe la base locale
    override fun observer(): Flow<List<Utilisateur>> =
        dao.observerTous().map { entities -> entities.map { it.toModel() } }

    // On rafraîchit depuis le réseau et on écrit en base
    override suspend fun rafraichir() {
        val distants = api.getUtilisateurs().map { it.toEntity() }
        dao.inserer(distants)
    }
}

```

Pourquoi l'offline-first

En faisant de la base locale la source de vérité, l'app reste utilisable hors-ligne, l'UI se met à jour automatiquement via le Flow, et le réseau devient un simple mécanisme de synchronisation. C'est l'architecture de référence des apps robustes.

5.5 Tester une application Compose

Deux grandes familles de tests :

- **Tests unitaires** (JVM, rapides) : logique des ViewModels, Repositories, mappers. On utilise JUnit, des coroutines de test et des faux Repositories.

- **Tests d'UI Compose** (instrumentés) : on interagit avec l'arbre de composables via les **sémantiques**, on cherche des nœuds et on vérifie l'affichage.

```
// Test unitaire d'un ViewModel
@Test
fun chargement_met_a_jour_letat() = runTest {
    val repo = FakeRepository(listOf(Utilisateur(1, "Alice")))
    val vm = AccueilViewModel(repo)

    vm.charger()

    assertEquals(1, vm.uiState.value.utilisateurs.size)
    assertFalse(vm.uiState.value.isLoading)
}

// Test d'UI Compose
@get:Rule val composeRule = createComposeRule()

@Test
fun le_compteur_sincrement() {
    composeRule.setContent { Compteur() }

    composeRule.onNodeWithText("Cliqué 0 fois").performClick()
    composeRule.onNodeWithText("Cliqué 1 fois").assertIsDisplayed()
}
```

Rendre l'UI testable

Ajoutez `Modifier.testTag("...")` sur les éléments à cibler, et privilégiez les composables **stateless** : ils se testent et se prévisualisent sans dépendances. Une UI pilotée par un état clair est une UI facile à tester.

5.6 Gérer proprement les erreurs

Plutôt que de laisser remonter des exceptions, encapsulez les résultats. Un type `Result` (scellé) rend les cas de succès et d'échec explicites et impossibles à ignorer.

```
sealed interface Resultat<out T> {
    data class Succes<T>(val donnee: T) : Resultat<T>
    data class Echec(val message: String) : Resultat<Nothing>
}

suspend fun chargerEnSecurite(): Resultat<List<Utilisateur>> = try {
    Resultat.Succes(api.getUtilisateurs().map { it.toModel() })
} catch (e: IOException) {
    Resultat.Echec("Pas de connexion réseau")
} catch (e: HttpException) {
    Resultat.Echec("Erreur serveur (${e.code()})")
}
```

5.7 Préférences modernes avec DataStore

`DataStore` remplace les anciennes `SharedPreferences` : il est asynchrone (coroutines), transactionnel et expose les données sous forme de `Flow`.

```

val Context.dataStore by preferencesDataStore(name = "reglages")
val CLE_THEME_SOMBRE = booleanPreferencesKey("theme_sombre")

// Lire (réactif)
val themeSombre: Flow<Boolean> = context.dataStore.data
    .map { prefs -> prefs[CLE_THEME_SOMBRE] ?: false }

// Écrire
suspend fun definirThemeSombre(aktif: Boolean) {
    context.dataStore.edit { prefs -> prefs[CLE_THEME_SOMBRE] = aktif }
}

```

5.8 Tâches en arrière-plan avec WorkManager

Pour un travail différé et **garanti** (synchronisation, envoi de fichiers) qui doit survivre à la fermeture de l'app, on utilise [WorkManager](#).

```

class SyncWorker(ctx: Context, params: WorkerParameters) :
    CoroutineWorker(ctx, params) {
    override suspend fun doWork(): Result {
        return try { repository.rafraichir(); Result.success() }
        catch (e: Exception) { Result.retry() }
    }
}

// Planifier toutes les 6 h, uniquement en réseau
val travail = PeriodicWorkRequestBuilder<SyncWorker>(6, TimeUnit.HOURS)
    .setConstraints(Constraints(requiredNetworkType = NetworkType.CONNECTED))
    .build()
WorkManager.getInstance(context).enqueue(travail)

```

5.9 Pagination de grandes listes (Paging 3)

Paging 3 charge les données page par page et s'intègre directement à [LazyColumn](#), avec gestion du chargement et des erreurs.

```

// ViewModel : flux paginé mis en cache
val utilisateurs: Flow<PagingData<Utilisateur>> =
    Pager(PagingConfig(pageSize = 20)) { UtilisateurPagingSource(api) }
    .flow.cachedIn(viewModelScope)

// UI
val lazyItems = viewModel.utilisateurs.collectAsLazyPagingItems()
LazyColumn {
    items(lazyItems.itemCount) { i ->
        lazyItems[i]?.let { Carte(it.nom, "ID ${it.id}") }
    }
    if (lazyItems.loadState.append is LoadState.Loading) {
        item { CircularProgressIndicator() }
    }
}

```

5.10 Catalogue d'opérateurs Flow

Les opérateurs transforment et combinent les flux. Les plus utiles au quotidien :

Opérateur	Effet
map / filter	Transformer / filtrer chaque valeur émise
combine	Fusionner plusieurs flux en un (ex. données + filtre)
debounce	Attendre une pause avant d'émettre (recherche en direct)
distinctUntilChanged	Ignorer les valeurs identiques consécutives
flatMapLatest	Annuler le flux précédent quand une nouvelle valeur arrive
catch	Intercepter une erreur dans le flux
stateIn / shareIn	Convertir un Flow froid en StateFlow/SharedFlow chaud

```
// Recherche réactive : on attend que l'utilisateur arrête de taper,
// on ignore les répétitions, et on relance la requête à chaque nouveau texte.
val resultats: StateFlow<List<Article>> = requete
    .debounce(300)
    .distinctUntilChanged()
    .flatMapLatest { texte -> repository.rechercher(texte) }
    .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
```

5.11 Injection de dépendances et faux objets en test

Pour tester un ViewModel sans vrai réseau ni base, on lui injecte un **faux Repository** (fake) qui retourne des données contrôlées. C'est la raison d'être de l'injection de dépendances.

```
// Interface partagée
interface ArticleRepository { suspend fun lister(): List<Article> }

// Implémentation réelle (production)
class ArticleRepositoryReseau(private val api: ApiService) : ArticleRepository {
    override suspend fun lister() = api.getArticles()
}

// Faux pour les tests : déterministe, sans réseau
class FakeArticleRepository(private val data: List<Article>) : ArticleRepository {
    override suspend fun lister() = data
}

@Test
fun chargement_expose_les_articles() = runTest {
    val vm = ArticleViewModel(FakeArticleRepository(listOf(Article(1, "Test"))))
    vm.charger()
    assertEquals(1, vm.uiState.value.articles.size)
}
```

5.12 Tests instrumentés & intégration continue

On distingue les tests **locaux** (sur la JVM, très rapides : [src/test](#)) des tests **instrumentés** (sur un appareil/émulateur : [src/androidTest](#)), qui incluent les tests d'UI Compose.

- Lancer tous les tests locaux : `./gradlew test`.

- Lancer les tests instrumentés : `./gradlew connectedAndroidTest` (appareil requis).
- **Intégration continue (CI)** : un service comme GitHub Actions exécute automatiquement les tests à chaque push, garantissant qu'une contribution ne casse rien.

Tester, gage de qualité

Même quelques tests bien choisis (la logique métier et un écran clé) augmentent nettement la fiabilité et la note. Intégrez-les dès le départ plutôt qu'à la fin.

5.13 Exercices — Partie 5

1. Définissez une interface Retrofit et un Repository qui expose un Flow de données.
2. Ajoutez Room : entité, DAO renvoyant un Flow, et synchronisation réseau → base (offline-first).
3. Écrivez un test unitaire du ViewModel avec un FakeRepository et runTest.
4. Écrivez un test d'UI vérifiant l'affichage de l'état de chargement puis de la liste.

Partie 6 — Bonnes pratiques & projet fil rouge

6.1 Checklist du développeur Compose

- Un composable = une responsabilité ; préférez de petits composables stateless.
- Remontez l'état (state hoisting) ; la source de vérité vit dans le ViewModel.
- Flux unidirectionnel : l'état descend, les événements remontent.
- Collectez l'état avec `collectAsStateWithLifecycle()`.
- Effets de bord uniquement via `LaunchedEffect`, `DisposableEffect`, etc.
- Clés stables dans les listes ; types immuables pour la stabilité.
- Thème centralisé Material 3 ; supportez le mode sombre dès le départ.
- Testez la logique (unit) et les écrans clés (UI tests).
- Mesurez la performance (Layout Inspector, rapports du compilateur, baseline profiles).

6.2 Projet fil rouge proposé : « MiniRéseau »

Pour consolider l'ensemble, construisez une petite application de répertoire d'utilisateurs réunissant tout le programme :

1. **Données** : API Retrofit + base Room, Repository offline-first.
2. **Architecture** : ViewModel par écran, UiState en sealed interface, Hilt pour l'injection.
3. **Navigation** : liste → détail (type-safe), avec un écran « réglages ».
4. **UI** : Material 3, thème clair/sombre, LazyColumn animée, états vide/chargement/erreur soignés.
5. **Avancé** : animation de transition liste→détail, jauge custom au Canvas dans le profil.
6. **Qualité** : tests unitaires des ViewModels et tests d'UI des écrans principaux.

Conseil de progression

Construisez le projet par tranches verticales : un écran de bout en bout (UI → ViewModel → Repository → données) avant de passer au suivant. Vous verrez tôt un résultat fonctionnel et chaque couche se renforcera naturellement.

6.3 Pour aller plus loin

- **Compose Multiplatform** : partager l'UI entre Android, iOS, desktop et web.
- **Paging 3** : pagination de grandes listes intégrée à LazyColumn.
- **WorkManager / DataStore** : tâches en arrière-plan et préférences modernes.
- **Accessibilité** : descriptions de contenu, tailles tactiles, contrastes.
- **CI/CD** : tests automatisés, captures d'écran (screenshot testing).

6.4 Pièges fréquents & anti-patterns Compose

Quelques erreurs courantes chez les débutants — et comment les éviter :

Anti-pattern	Bonne pratique
Effet de bord dans le corps d'un composable	Encadrer avec <code>LaunchedEffect</code> / <code>rememberCoroutineScope</code>
État stocké dans une variable normale	Utiliser <code>remember</code> + <code>mutableStateOf</code>
Logique métier dans le composable	Déplacer dans le <code>ViewModel</code>
Column défilante pour une longue liste	Utiliser <code>LazyColumn</code> avec des clés
Texte affiché codé en dur	Externaliser dans <code>strings.xml</code>
Lire <code>collectAsState()</code> sans cycle de vie	Utiliser <code>collectAsStateWithLifecycle()</code>
Recréer des objets coûteux à chaque recomposition	Les envelopper dans <code>remember</code>

Reconnaître pour corriger

La plupart de ces pièges viennent d'une confusion entre « ce qui décrit l'UI » (le composable, pur et rapide) et « ce qui agit » (effets, `ViewModel`). Garder cette frontière nette résout la majorité des problèmes.

6.5 Glossaire

Terme	Définition
Composable	Fonction <code>@Composable</code> qui décrit une portion d'UI
Recomposition	Ré-exécution d'un composable quand l'état qu'il lit change
State hoisting	Remontée de l'état vers l'appelant pour des composables <code>stateless</code>
UDF	Flux de données unidirectionnel : état descendant, événements remontants
Modifier	Décorateur chaînable (taille, marge, fond, clic...)
BOM	Bill of Materials : catalogue de versions <code>Compose</code> cohérentes
<code>StateFlow</code>	Flux observable détenant toujours une valeur courante
Side effect	Action interagissant avec l'extérieur, encadrée par un effet
Stable / immuable	Type dont <code>Compose</code> peut sauter la recomposition en toute sûreté

Versions de référence (édition 2026) : `Compose BOM 2026.05.x`, artefacts core **1.11.x**, `Material 3 1.4.x`, `Kotlin 2.x`. Vérifiez toujours la version stable la plus récente sur le site officiel des développeurs Android.

Partie 7 — Étude de cas : application Calculatrice

Cette étude de cas met en pratique l'ensemble du cours sur un projet complet et réaliste : une **application calculatrice** écrite en Kotlin et Jetpack Compose, structurée selon le flux de données unidirectionnel. Le projet Android Studio complet est fourni séparément (archive [Calculatrice-AndroidStudio-Projet.zip](#)), accompagné d'une **simulation interactive** jouable dans un navigateur.

7.1 Cahier des charges

L'application doit permettre les opérations d'une calculatrice de base, avec une interface tactile soignée :

- Saisir des nombres (chiffres et virgule décimale) et enchaîner les opérations $+$ $-$ $\times$ $\div$.
- Respecter la priorité des opérateurs ($\times$ et $\div$ avant $+$ et $-$).
- Afficher l'expression en cours et un aperçu du résultat en temps réel.
- Fournir les fonctions **C** (tout effacer), **X** (effacer un caractère) et **±** (changer de signe).
- Gérer les erreurs (division par zéro, expression invalide) sans planter.
- Proposer un thème Material 3 clair/sombre avec couleur dynamique.

7.2 Architecture du projet

Le projet suit l'architecture recommandée vue en Partie 3 : une UI Compose sans logique, un **ViewModel** qui détient l'état, et un moteur de calcul isolé et testable.

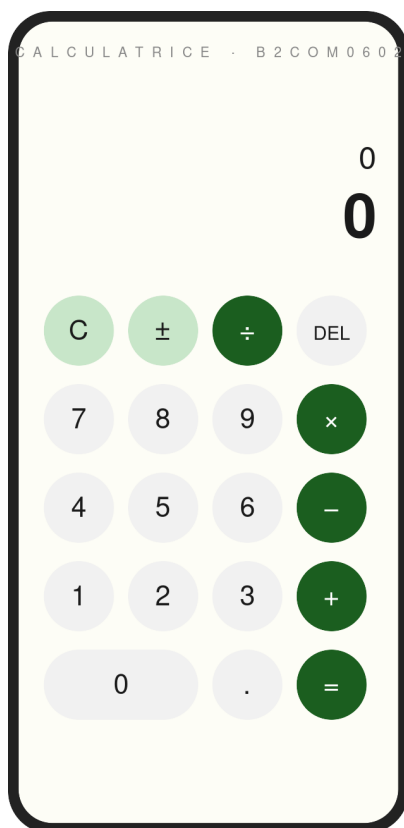
Fichier	Rôle
MainActivity.kt	Point d'entrée ; installe le thème et l'écran Compose
logic/Evaluateur.kt	Moteur de calcul (tokeniser $\rightarrow$ shunting-yard $\rightarrow$ évaluation)
ui/CalculatriceViewModel.kt	État (CalculatriceState) et logique (CalcAction)
ui/CalculatriceEcran.kt	Interface Compose : affichage + clavier
ui/theme/*	Thème Material 3 (couleurs, typographie, clair/sombre)

Flux unidirectionnel (rappel)

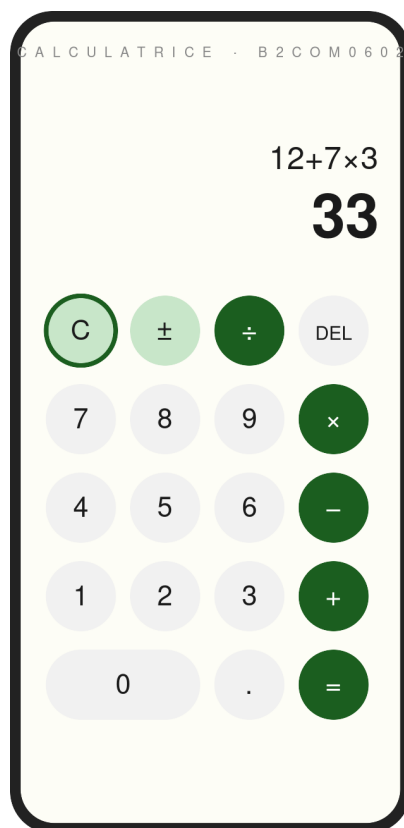
L'écran envoie des CalcAction au ViewModel (« events up »). Le ViewModel recalcule le CalculatriceState (« state down »). Compose recompose l'affichage. Le moteur Evaluateur ne connaît ni l'UI ni le ViewModel : on peut le tester isolément.

7.3 Aperçu de l'application

Voici l'interface telle qu'elle s'affiche sur un téléphone : l'état initial, et le résultat d'un calcul respectant la priorité des opérateurs ($12 + 7 \times 3 = 33$).



État initial

 $12 + 7 \times 3 = 33$

Simulation interactive

Le fichier [Calculatrice-Simulation.html](#) reproduit fidèlement l'application (même logique que [Evalueur.kt](#)). Ouvrez-le dans n'importe quel navigateur pour tester la calculatrice immédiatement, sans rien installer.

7.4 Le moteur de calcul (Evalueur.kt)

Le cœur de l'application. Il transforme la chaîne saisie en jetons, la convertit en notation polonaise inverse (algorithme du **shunting-yard**) pour respecter la priorité, puis l'évalue.

```
object Evalueur {

    fun evaluer(expression: String): Double {
        val jetons = tokeniser(expression)
        if (jetons.isEmpty()) return 0.0
        return evaluerRpn(versRpn(jetons))
    }

    // Découpe en nombres/opérateurs, gère le moins unaire (-5, 3×-2)
    private fun tokeniser(entree: String): List<String> {
        val s = entree.replace("x", "*").replace("÷", "/").replace("-", "-").replace(" ", "")
        val jetons = mutableListOf<String>()
        var i = 0; var attendOperande = true
        while (i < s.length) {
```

```

        val c = s[i]
        when {
            c.isDigit() || c == '.' -> {
                val sb = StringBuilder()
                while (i < s.length && (s[i].isDigit() || s[i] == '.')) { sb.append(s[i]);
i++ }

                jetons.add(sb.toString()); attendOperande = false
            }
            c in "+-*/" -> {
                if (c == '-' && attendOperande) {
                    val sb = StringBuilder("-"); i++
                    while (i < s.length && (s[i].isDigit() || s[i] == '.')) {
sb.append(s[i]); i++ }
                    if (sb.length > 1) { jetons.add(sb.toString()); attendOperande = false
}
                    } else { jetons.add(c.toString()); i++; attendOperande = true }
                }
            }
            else -> i++
        }
    }
    return jetons
}

private fun priorite(op: String) = when (op) { "+", "-" -> 1; "*", "/" -> 2; else -> 0 }

private fun versRpn(jetons: List<String>): List<String> {
    val sortie = mutableListOf<String>(); val ops = ArrayDeque<String>()
    for (j in jetons) {
        if (j.toDoubleOrNull() != null) sortie.add(j)
        else {
            while (ops.isNotEmpty() && priorite(ops.last()) >= priorite(j))
sortie.add(ops.removeLast())
            ops.addLast(j)
        }
    }
    while (ops.isNotEmpty()) sortie.add(ops.removeLast())
    return sortie
}

private fun evaluerRpn(rpn: List<String>): Double {
    val pile = ArrayDeque<Double>()
    for (j in rpn) {
        val n = j.toDoubleOrNull()
        if (n != null) pile.addLast(n)
        else {
            val b = pile.removeLast(); val a = pile.removeLast()
            pile.addLast(when (j) {
                "+" -> a + b; "-" -> a - b; "*" -> a * b
                "/" -> if (b == 0.0) throw ArithmeticException("Division par zéro") else a
/ b
            })
            else -> throw IllegalArgumentException("Opérateur inconnu")
        })
    }
    return pile.last()
}
}

```

7.5 L'état et la logique (CalculatriceViewModel.kt)

```
data class CalculatriceState(
    val expression: String = "",
    val resultat: String = "0"
)

sealed interface CalcAction {
    data class Chiffre(val c: Char) : CalcAction
    data class Operateur(val op: Char) : CalcAction
    data object Decimale : CalcAction
    data object Egal : CalcAction
    data object Effacer : CalcAction
    data object Supprimer : CalcAction
    data object Signe : CalcAction
}

class CalculatriceViewModel : ViewModel() {
    var state by mutableStateOf(CalculatriceState()); private set
    private val operateurs = setOf('+', '-', 'x', '÷')

    fun onAction(action: CalcAction) = when (action) {
        is CalcAction.Chiffre -> ajouterChiffre(action.c)
        is CalcAction.Operateur -> ajouterOperateur(action.op)
        CalcAction.Decimale -> ajouterDecimale()
        CalcAction.Egal -> calculer()
        CalcAction.Effacer -> state = CalculatriceState()
        CalcAction.Supprimer -> supprimer()
        CalcAction.Signe -> changerSigne()
    }

    private fun calculer() {
        if (state.expression.isEmpty()) return
        try {
            val texte = Evalueur.formater(Evalueur.evaluer(state.expression))
            state = CalculatriceState(expression = texte, resultat = texte)
        } catch (_: ArithmeticException) { state = state.copy(resultat = "Division par zéro") }

        catch (_: Exception) { state = state.copy(resultat = "Erreur") }
    }

    // ajouterChiffre, ajouterOperateur, ajouterDecimale, supprimer, changerSigne, aperçu :
    // voir le projet complet (gèrent les garde-fous : pas deux opérateurs de suite, un seul
    point...)
}
```

Aperçu en direct

À chaque saisie, le ViewModel tente d'évaluer l'expression (méthode `aperçu()`) et met à jour le résultat sans bloquer si l'expression est incomplète. C'est un exemple concret d'**état dérivé**.

7.6 L'interface Compose (CalculatriceEcran.kt)

```
@Composable
fun CalculatriceEcran(viewModel: CalculatriceViewModel = viewModel()) {
    val state = viewModel.state
    val lignes = listOf(
```

```

        listOf("C", "±", "÷", "⊗"),
        listOf("7", "8", "9", "x"),
        listOf("4", "5", "6", "-"),
        listOf("1", "2", "3", "+"),
        listOf("0", ".", "=")
    )
}
Surface(Modifier.fillMaxSize(), color = MaterialTheme.colorScheme.background) {
    Column(Modifier.fillMaxSize().padding(16.dp)) {
        Box(Modifier.fillMaxWidth().weight(1f), contentAlignment = Alignment.BottomEnd) {
            Column(horizontalAlignment = Alignment.End) {
                Text(state.expression.ifEmpty { "0" }, fontSize = 32.sp,
                    color = MaterialTheme.colorScheme.onBackground.copy(alpha = 0.6f))
                Text(state.resultat, fontSize = 56.sp, maxLines = 1)
            }
        }
        lignes.forEach { ligne ->
            Row(Modifier.fillMaxWidth().padding(vertical = 4.dp),
                horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                ligne.forEach { libelle ->
                    Touche(libelle,
                        Modifier.weight(if (libelle == "0") 2f else 1f)
                            .aspectRatio(if (libelle == "0") 2f else 1f)) {
                        viewModel.onAction(actionPour(libelle))
                    }
                }
            }
        }
    }
}

```

7.7 Le point d'entrée (MainActivity.kt)

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            CalculatriceTheme {
                CalculatriceEcran()
            }
        }
    }
}

```

7.8 Compiler et générer l'APK téléchargeable

L'archive [Calculatrice-AndroidStudio-Projet.zip](#) contient le projet complet, prêt à compiler.

Procédure :

7. Décompressez l'archive, puis dans Android Studio : **File** → **Open** et sélectionnez le dossier [calculatrice-compose](#).
8. Laissez Gradle se synchroniser (premier téléchargement des dépendances Compose ; connexion requise).
9. Pour exécuter : sélectionnez un émulateur ou un téléphone et cliquez sur **Run** .

10. Pour générer l'APK : menu **Build** → **Build Bundle(s) / APK(s)** → **Build APK(s)**.
11. L'APK est produit dans `app/build/outputs/apk/debug/app-debug.apk` — c'est le fichier installable sur un téléphone Android.
12. En ligne de commande (équivalent) : `./gradlew assembleDebug` pour un APK de debug, `./gradlew assembleRelease` pour une version à signer.

Installer l'APK sur un téléphone

Transférez `app-debug.apk` sur l'appareil, autorisez « installer des applications de sources inconnues », puis ouvrez le fichier. Pour une distribution publique (Play Store), il faut générer un APK/AAB **release signé** avec une clé de signature.

Note sur le présent support

L'APK n'a pas été pré-compilé dans cet environnement (le SDK Android et les serveurs de téléchargement de Google n'y sont pas accessibles). Le projet fourni est néanmoins complet et se compile en un clic dans Android Studio selon la procédure ci-dessus, produisant l'APK téléchargeable.

7.9 Tester la calculatrice

Le moteur `Evaluateur` est isolé de l'UI : il se teste donc très facilement par des tests unitaires (rapides, sur la JVM). C'est exactement l'intérêt d'une bonne architecture.

```
class EvaluateurTest {
    @Test fun priorite_des_operateurs() {
        assertEquals(33.0, Evaluateur.evaluer("12+7×3"), 0.0001)
    }
    @Test fun division_decimale() {
        assertEquals(2.5, Evaluateur.evaluer("10÷4"), 0.0001)
    }
    @Test fun moins_unaire() {
        assertEquals(-18.0, Evaluateur.evaluer("9×-2"), 0.0001)
    }
    @Test(expected = ArithmeticException::class)
    fun division_par_zero_leve_une_exception() {
        Evaluateur.evaluer("5÷0")
    }
}
```

On peut compléter par un test d'UI Compose vérifiant qu'un appui sur les touches met bien à jour l'affichage (via `createComposeRule` et `onNodeWithText(...).performClick()`).

Le test comme filet de sécurité

Avec ces quelques tests, toute modification future du moteur qui casserait un calcul serait immédiatement détectée. C'est ce filet qui permet de refactoriser et d'évoluer sereinement — un réflexe à reproduire dans votre projet.

7.10 Pistes d'amélioration (pour aller plus loin)

- Ajouter un historique des calculs (LazyColumn) persisté avec Room ou DataStore.

- Gérer les parenthèses et les pourcentages dans le moteur Evalueur.
- Écrire des tests unitaires du moteur (JUnit) et des tests d'UI du clavier (createComposeRule).
- Ajouter une calculatrice scientifique (sin, cos, racine, puissance) en mode paysage.
- Animer les touches et le changement de résultat (animateContentSize, AnimatedContent).

Partie 8 — Gestion de projet, qualité & expérience utilisateur

Réussir une application en équipe demande plus que des compétences techniques : il faut **organiser** le travail, garantir la **qualité**, soigner **l'expérience utilisateur** et **documenter**. Cette partie outille la dimension « conduite de projet » attendue dans le projet tuteuré.

8.1 Méthodes de gestion de projet

Deux grandes familles s'opposent. Le **cycle en V (ou cascade)** enchaîne les phases de façon linéaire (analyse → conception → développement → tests). Simple, mais rigide : un changement tardif coûte cher. Les **méthodes agiles** (Scrum, Kanban) procèdent par **itérations courtes**, livrant régulièrement un produit partiel testable et s'adaptant aux retours.

Pour un projet étudiant de quelques semaines, une approche **agile allégée** est idéale :

- **Backlog** : la liste priorisée de toutes les fonctionnalités à réaliser (sous forme de user stories).
- **Sprints** : des cycles courts (1 à 2 semaines) au bout desquels une partie de l'app fonctionne.
- **Daily / point d'équipe** : un court échange régulier — fait / à faire / blocages.
- **Revue & rétrospective** : en fin de sprint, on montre ce qui marche et on améliore la façon de travailler.

Outils gratuits utiles

Un tableau Kanban (Trello, GitHub Projects, ClickUp) suffit à visualiser les tâches en colonnes « À faire / En cours / Fait ». Couplé à Git, il rend l'avancement visible de toute l'équipe et de l'enseignant.

8.2 Les user stories

Une **user story** décrit un besoin du point de vue de l'utilisateur, dans un format simple :

```
En tant que <type d'utilisateur>,  
je veux <faire quelque chose>  
afin de <obtenir un bénéfice>.
```

Exemple :

```
En tant qu'utilisateur, je veux enregistrer mes calculs récents  
afin de pouvoir les retrouver plus tard.
```

Chaque story est accompagnée de **critères d'acceptation** (conditions à remplir pour la considérer terminée). Elles découpent le projet en morceaux livrables et testables.

8.3 Le travail en équipe avec Git

Git est le socle de la collaboration. Un **workflow** simple et efficace pour une équipe de 3-4 :

13. La branche **main** contient toujours une version stable.
14. Chaque fonctionnalité se développe sur une **branche dédiée** (**feature/calcul-historique**).
15. Quand c'est prêt, on ouvre une **pull/merge request** relue par un binôme avant fusion dans **main**.

16. On récupère souvent les changements des autres ([pull](#)) pour limiter les conflits.

Messages de commit utiles

Écrivez des messages clairs et à l'impératif : [Ajoute l'écran d'historique](#), [Corrige la division par zéro](#). Un bon historique Git est une documentation vivante du projet et un critère de qualité.

8.4 La qualité du code

La qualité ne se voit pas à l'écran mais conditionne la maintenabilité et la note. Quelques principes :

- **Lisibilité** : noms explicites, fonctions courtes, code commenté là où c'est utile.
- **Architecture claire** : séparer UI, logique (ViewModel) et données (Repository) — voir Partie 3.
- **DRY** (Don't Repeat Yourself) : factoriser le code dupliqué.
- **Tests** : couvrir la logique métier et les écrans clés (Partie 5).
- **Style cohérent** : appliquer le style Kotlin officiel (reformatage automatique).
- **Revue de code** : relire les contributions des coéquipiers avant fusion.

8.5 L'expérience utilisateur (UX/UI)

Une application n'est utile que si elle est **agréable et facile à utiliser**. Quelques règles d'or de conception :

- **Clarté** : un écran = un objectif principal ; hiérarchiser l'information.
- **Cohérence** : mêmes couleurs, typographies et composants partout (un thème Material 3 centralisé).
- **Feedback** : indiquer le chargement, confirmer les actions, signaler les erreurs clairement.
- **Tolérance à l'erreur** : confirmations pour les actions destructrices, possibilité d'annuler.
- **Accessibilité** : contrastes suffisants, cibles tactiles ≥ 48 dp, descriptions pour les lecteurs d'écran.
- **Maquettes d'abord** : dessinez les écrans (papier, Figma) avant de coder — on corrige bien moins cher sur une maquette.

États de l'interface

Pour chaque écran, prévoyez les états : vide (aucune donnée), chargement, succès, et erreur. Une interface qui gère ces quatre cas paraît immédiatement professionnelle.

8.6 La documentation du projet

Documenter, c'est rendre le travail compréhensible et réutilisable. Les livrables documentaires attendus :

Document	Contenu
README	Présentation du projet, technologies, comment compiler et lancer
Manuel d'utilisation	Comment se servir de l'application (captures à l'appui)
Rapport de projet	Besoin, conception, architecture, choix techniques, difficultés, bilan

Document	Contenu
Commentaires de code	Expliquer le « pourquoi » des passages non triviaux

La documentation se construit en continu

N'attendez pas la veille du rendu. Tenez le README à jour, prenez des captures au fil de l'eau, notez les décisions importantes. Le rapport final n'en sera que plus facile à écrire.

8.7 Estimer et gérer les risques

Un projet réaliste anticipe ce qui peut mal tourner. Identifiez tôt les **risques** et prévoyez une parade.

Risque	Parade
Sous-estimation du temps	Découper en petites tâches, garder une marge, prioriser le minimum viable
Membre indisponible	Documenter, partager le code (Git), éviter les silos de connaissance
Difficulté technique imprévue	Prototyper tôt les parties risquées, demander de l'aide rapidement
Conflits Git / perte de code	Commiter souvent, pousser régulièrement, branches par fonctionnalité
Périmètre trop ambitieux	Définir tôt le « must have » vs « nice to have »

La règle du « bout en bout d'abord »

Faites fonctionner une fonctionnalité complète (UI → logique → données) le plus tôt possible, même minimale. Vous validez ainsi toute la chaîne technique et réduisez le risque le plus dangereux : découvrir un blocage la veille du rendu.

Partie 9 — Le développement multiplateforme

Nous apprenons le natif Android, mais il est essentiel de connaître le **paysage** des autres approches : un informaticien doit savoir choisir la bonne technologie selon le contexte. Cette partie compare les principales solutions multiplateformes.

9.1 Pourquoi le multiplateforme ?

Développer deux applications natives (Android + iOS) coûte cher : deux équipes, deux bases de code, deux fois la maintenance. Les frameworks multiplateformes visent à **partager un maximum de code** tout en restant proches du natif. Le compromis se joue entre **partage de code**, **performances** et **accès au natif**.

9.2 Flutter

Créé par Google, **Flutter** utilise le langage **Dart** et dessine lui-même toute l'interface via son propre moteur de rendu. Une seule base de code cible Android, iOS, web et desktop.

- **Forces** : rendu très fluide et identique partout, productivité élevée, riche catalogue de widgets, hot reload.
- **Limites** : taille d'app un peu plus grande ; look « maison » (à thématiser pour coller à chaque plateforme) ; langage Dart à apprendre.

9.3 React Native

React Native (Meta) permet d'écrire l'application en **JavaScript/TypeScript** avec le paradigme React, en pilotant de vrais composants natifs.

- **Forces** : réutilise l'immense écosystème JavaScript/React, large communauté, partage de compétences avec le web.
- **Limites** : le « pont » JS/natif peut limiter les performances sur les cas exigeants ; dépendance à des bibliothèques tierces pour certaines fonctions.

9.4 Kotlin Multiplatform (KMP) & Compose Multiplatform

Kotlin Multiplatform partage la **logique métier** (réseau, base de données, règles) en Kotlin entre Android et iOS, tout en laissant la possibilité d'une UI native. **Compose Multiplatform** étend même Jetpack Compose à iOS, desktop et web.

- **Forces** : on capitalise directement sur ce que vous apprenez ici (Kotlin, Compose) ; partage de la logique sans sacrifier le natif.
- **Limites** : écosystème plus jeune que Flutter/RN pour l'UI iOS (en maturation rapide).

9.5 Comment choisir ?

Critère	Natif (Android/iOS)	Flutter	React Native	KMP
Performances	Excellentes	Très bonnes	Bonnes	Excellentes (UI native)
Partage de code	Aucun	Très élevé	Élevé	Logique partagée
Accès au natif	Total	Bon (plugins)	Bon (modules)	Total
Courbe d'apprentissage	2 langages	Dart	JS/React	Kotlin
Idéal pour	Apps exigeantes	MVP multi-OS rapides	Équipes web/React	Apps Kotlin évolutives

Le natif reste la meilleure école

Apprendre le natif Android donne les fondations (cycle de vie, UI déclarative, threads, état) qui rendent ensuite n'importe quel framework multiplateforme bien plus facile à comprendre. C'est un investissement durable.

Partie 10 — Déploiement & publication

Une application n'a de valeur que lorsqu'elle atteint ses utilisateurs. Cette partie couvre la dernière phase du cycle de vie : produire un livrable, le signer, le publier et en assurer le suivi.

10.1 APK et AAB

Deux formats de livraison Android :

- **APK** (Android Package) : le format installable directement sur un appareil. Parfait pour les tests et la distribution directe.
- **AAB** (Android App Bundle) : le format de publication sur le Play Store. Google génère ensuite des APK optimisés pour chaque appareil.

```
# En ligne de commande (depuis la racine du projet)
./gradlew assembleDebug      # APK de debug (tests)
./gradlew assembleRelease    # APK de release (à signer)
./gradlew bundleRelease      # AAB de release (Play Store)
```

10.2 La signature de l'application

Tout APK installé est **signé** numériquement. En debug, Android Studio utilise une clé de debug automatique. Pour une publication, vous devez créer votre propre **clé de signature** (keystore) et la garder précieusement — elle identifie durablement votre application.

17. Menu **Build** → **Generate Signed Bundle / APK...**
18. Créer un **keystore** (fichier .jks) et un mot de passe (à conserver en lieu sûr).
19. Choisir la variante **release** et générer l'AAB/APK signé.

Ne perdez jamais votre keystore

Sans la clé d'origine, vous ne pourrez plus mettre à jour une application déjà publiée. Sauvegardez le keystore et ses mots de passe en lieu sûr (gestionnaire de secrets, coffre partagé d'équipe).

10.3 Le versioning

Chaque publication porte deux numéros, définis dans `build.gradle.kts` :

- **versionCode** : un entier qui augmente à chaque version (utilisé par le système pour les mises à jour).
- **versionName** : la version lisible par l'utilisateur (ex. "1.2.0"), souvent en versionnage sémantique **MAJEUR.MINEUR.CORRECTIF**.

10.4 Publier sur le Google Play Store

20. Créer un compte développeur **Google Play Console** (inscription payante unique).
21. Créer une fiche d'application : titre, description, captures d'écran, icône, catégorie, politique de confidentialité.
22. Téléverser l'**AAB signé** sur une piste (test interne, fermé, ouvert, puis production).

23. Renseigner le classement de contenu et la conformité, puis soumettre à la **revue** de Google.

24. Après validation, l'application est **publiée** et disponible au téléchargement.

Autres canaux de distribution

Hors Play Store : distribution directe de l'APK (sites, QR code), magasins alternatifs (Galaxy Store, Amazon Appstore), ou plateformes de test comme Firebase App Distribution pour partager des builds avec un groupe de testeurs.

10.5 Le suivi après publication

Le travail ne s'arrête pas à la publication. La phase de **suivi (maintenance)** assure la pérennité de l'application :

- **Suivi des plantages** : un outil comme **Firebase Crashlytics** remonte automatiquement les erreurs rencontrées par les utilisateurs.
- **Analytique d'usage** : **Firebase/Google Analytics** mesure l'utilisation (écrans visités, rétention) pour prioriser les améliorations.
- **Retours utilisateurs** : avis et notes du Play Store, à analyser et à traiter.
- **Mises à jour** : corrections de bugs, nouvelles fonctionnalités, adaptation aux nouvelles versions d'Android.

La boucle d'amélioration continue

Mesurer → comprendre → améliorer → republier : c'est ce cycle qui distingue un produit vivant d'un simple exercice. Même à l'échelle d'un projet étudiant, anticiper le suivi montre une vraie maturité professionnelle.

10.6 Préparer la fiche de l'application (store listing)

Publier ne se résume pas au fichier : il faut une **fiche** attractive et conforme. Les éléments à préparer :

Élément	Recommandation
Icône	512×512 px, lisible en petit, cohérente avec l'identité de l'app
Nom & description courte	Clairs, avec les mots-clés essentiels
Description complète	Bénéfices et fonctionnalités, sans promesses trompeuses
Captures d'écran	Plusieurs écrans représentatifs (téléphone et tablette)
Bandeau / vidéo	Présentation visuelle (optionnelle mais valorisante)
Classement de contenu	Questionnaire obligatoire (âge, contenu)
Politique de confidentialité	Obligatoire si collecte de données personnelles

Soigner la première impression

La fiche est la vitrine de l'application : icône, captures et description déterminent le téléchargement. Pour le projet, préparez une icône propre et 3 à 5 captures soignées — c'est aussi un excellent support de soutenance.

Partie 11 — Le projet tuteuré : organisation & évaluation

Cette partie est le **mode d'emploi** du projet à réaliser. Elle précise l'organisation, propose un cahier des charges type, un planning, la liste des livrables et la grille d'évaluation. C'est le document de référence à relire régulièrement pendant le projet.

11.1 Énoncé du projet

Par **groupes de trois ou quatre**, réalisez une **application mobile fonctionnelle** répondant à un cahier des charges. Le sujet est soit **imposé par l'enseignant**, soit **proposé par l'équipe puis validé**. L'application doit être développée en **Android natif (Kotlin / Jetpack Compose)** et couvrir le cycle de vie : analyse, conception, développement, tests, et préparation au déploiement.

11.2 Exemples de sujets

Si votre équipe propose un sujet, visez une application réaliste, utile et de périmètre maîtrisable. Quelques idées :

Application	Fonctionnalités clés
Gestionnaire de tâches (To-Do)	Ajouter/cocher/supprimer des tâches, catégories, persistance locale
Carnet de dépenses	Saisir des dépenses, totaux par catégorie, graphique simple
Bloc-notes / Mémo	Créer/éditer des notes, recherche, sauvegarde Room
Météo	Ville recherchée, appel d'une API, affichage des prévisions
Quiz / Flashcards	Questions, score, révision espacée
Annuaire / Catalogue	Liste → détail, recherche, données locales ou réseau

Bien dimensionner le périmètre

Mieux vaut une application simple mais soignée, complète et sans bug, qu'une application ambitieuse mais inachevée. Définissez un cœur fonctionnel (le « minimum viable ») et gardez les options pour la fin si le temps le permet.

11.3 Le cahier des charges (modèle)

Chaque équipe rédige un cahier des charges en début de projet. Il doit contenir au minimum :

25. **Contexte et objectif** : à quoi sert l'application, pour qui.
26. **Utilisateurs cibles** : qui va l'utiliser et dans quel contexte.
27. **Fonctionnalités** : la liste, séparée en « indispensables » et « optionnelles ».
28. **User stories** : les besoins exprimés du point de vue utilisateur.
29. **Contraintes techniques** : Android natif, versions minimales, données locales/réseau.
30. **Maquettes** : esquisses des écrans principaux.

31.Planning prévisionnel : jalons et répartition des tâches.

11.4 Planning et jalons

Un découpage en jalons (milestones) rythme le projet et facilite le suivi :

Jalon	Échéance	Livrable attendu
J1 · Cadrage	Semaine 1	Cahier des charges + maquettes + dépôt Git créé
J2 · Squelette	Semaine 2	Navigation et écrans vides, architecture en place
J3 · Cœur fonctionnel	Semaine 3-4	Fonctionnalités indispensables opérationnelles
J4 · Données & tests	Semaine 5	Persistance/réseau + premiers tests
J5 · Finition & APK	Semaine 6	UI soignée, APK signé, documentation
J6 · Soutenance	Semaine 7	Démonstration + rapport + code final

Suivi de l'avancement

Tenez un tableau Kanban à jour et committez régulièrement : l'enseignant peut ainsi suivre la progression réelle de chaque membre. Un dépôt actif et régulier est un signe de bonne gestion de projet.

11.5 Livrables attendus

Livrable	Description
Code source (Git)	Dépôt complet, historique propre, branches et commits réguliers
APK installable	Application compilée et signée, prête à être testée
Cahier des charges	Document de cadrage validé
Rapport de projet	Besoin, conception, architecture, choix, difficultés, bilan
Manuel d'utilisation	Guide d'usage avec captures d'écran
Support de soutenance	Diaporama de présentation pour la démonstration

11.6 Grille d'évaluation

L'évaluation valorise l'ensemble de la démarche, pas seulement le résultat technique :

Critère	Poids	Indicateurs
Fonctionnalité & conformité	25 %	Respect du cahier des charges, absence de plantage
Qualité du code & architecture	20 %	Lisibilité, structure UI/ViewModel/Données, Git
Expérience utilisateur (UX/UI)	15 %	Ergonomie, cohérence, accessibilité, états gérés
Tests	10 %	Pertinence et couverture des tests
Gestion de projet & autonomie	15 %	Planning tenu, répartition, progression régulière

Critère	Poids	Indicateurs
Documentation	10 %	README, manuel, rapport
Soutenance	5 %	Clarté de la démo et des réponses aux questions

Conseils pour réussir

Commencez tôt et committez souvent ; faites fonctionner un petit bout de bout en bout avant d'élargir ; testez sur un vrai appareil ; soignez les détails visuels en fin de parcours ; préparez et répétez la démonstration. La régularité bat toujours le sprint de dernière minute.

11.7 Du modèle au projet : réutiliser la calculatrice

L'étude de cas « Calculatrice » (Partie 7) est votre **patron de référence** : architecture en couches, ViewModel, état, UI Compose, gestion d'erreurs, thème, tests et génération d'APK. Inspirez-vous-en pour structurer votre propre application — la base technique y est entièrement illustrée et le projet fourni se compile en un clic.

Partie 12 — Étude de cas guidée : application To-Do

Cette seconde étude de cas est un **squelette guidé** d'une application de gestion de tâches (To-Do). Plus proche d'un sujet de projet que la calculatrice, elle montre comment structurer une application « liste + ajout + persistance ». Servez-vous-en comme point de départ pour votre propre projet de groupe.

12.1 Cahier des charges résumé

- Afficher la liste des tâches.
- Ajouter une nouvelle tâche (saisie de texte).
- Cocher / décocher une tâche (terminée ou non).
- Supprimer une tâche.
- Conserver les tâches après fermeture (persistance locale).

12.2 Le modèle de données

```
// Modèle métier
data class Tache(
    val id: Long,
    val libelle: String,
    val terminee: Boolean = false
)
```

12.3 L'état et la logique (ViewModel)

```
data class TodoUiState(
    val taches: List<Tache> = emptyList(),
    val saisie: String = ""
)

class TodoViewModel : ViewModel() {
    var state by mutableStateOf(TodoUiState()); private set
    private var prochainId = 1L

    fun onSaisieChange(texte: String) {
        state = state.copy(saisie = texte)
    }

    fun ajouter() {
        val libelle = state.saisie.trim()
        if (libelle.isEmpty()) return
        val nouvelle = Tache(id = prochainId++, libelle = libelle)
        state = state.copy(taches = state.taches + nouvelle, saisie = "")
    }

    fun basculer(id: Long) {
        state = state.copy(taches = state.taches.map {
            if (it.id == id) it.copy(terminee = !it.terminee) else it
        })
    }
}
```

```

fun supprimer(id: Long) {
    state = state.copy(taches = state.taches.filterNot { it.id == id })
}

```

Listes immuables et copy

Remarquez qu'on ne modifie jamais la liste en place : on en crée une nouvelle avec `+`, `map` ou `filterNot`. C'est le style attendu par Compose pour détecter le changement et recomposer.

12.4 L'interface (Compose)

```

@Composable
fun TodoEcran(viewModel: TodoViewModel = viewModel()) {
    val state = viewModel.state
    Scaffold(topBar = { TopAppBar(title = { Text("Mes tâches") }) }) { padding ->
        Column(Modifier.padding(padding).padding(16.dp)) {

            // Zone de saisie + bouton Ajouter
            Row(verticalAlignment = Alignment.CenterVertically) {
                OutlinedTextField(
                    value = state.saisie,
                    onChange = viewModel::onSaisieChange,
                    label = { Text("Nouvelle tâche") },
                    modifier = Modifier.weight(1f)
                )
                Spacer(Modifier.width(8.dp))
                Button(onClick = viewModel::ajouter) { Text("Ajouter") }
            }

            Spacer(Modifier.height(12.dp))

            if (state.taches.isEmpty()) {
                Text("Aucune tâche pour le moment.",
                    color = MaterialTheme.colorScheme.onSurfaceVariant)
            } else {
                LazyColumn(verticalArrangement = Arrangement.spacedBy(4.dp)) {
                    items(state.taches, key = { it.id }) { tache ->
                        LigneTache(
                            tache = tache,
                            onCheck = { viewModel.basculer(tache.id) },
                            onDelete = { viewModel.supprimer(tache.id) }
                        )
                    }
                }
            }
        }
    }
}

```

```

@Composable
private fun LigneTache(tache: Tache, onCheck: () -> Unit, onDelete: () -> Unit) {
    Row(verticalAlignment = Alignment.CenterVertically,
        modifier = Modifier.fillMaxWidth()) {
        Checkbox(checked = tache.terminee, onCheckedChange = { onCheck() })
        Text(

```

```

        text = tache.libelle,
        modifier = Modifier.weight(1f),
        textDecoration = if (tache.terminee) TextDecoration.LineThrough else null
    )
    IconButton(onClick = onDelete) {
        Icon(Icons.Default.Delete, contentDescription = "Supprimer")
    }
}
}

```

12.5 Ajouter la persistance (Room)

Pour conserver les tâches après fermeture, remplacez la liste en mémoire par une base **Room** (Partie 5). Le schéma : une entité `TacheEntity`, un `TacheDao` qui expose un `Flow<List<TacheEntity>>`, et un `Repository` qui relie le ViewModel à la base.

```

@Entity(tableName = "taches")
data class TacheEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val libelle: String,
    val terminee: Boolean = false
)

@Dao
interface TacheDao {
    @Query("SELECT * FROM taches ORDER BY id DESC")
    fun observer(): Flow<List<TacheEntity>>

    @Insert suspend fun inserer(t: TacheEntity)
    @Update suspend fun mettreAJour(t: TacheEntity)
    @Delete suspend fun supprimer(t: TacheEntity)
}

```

Votre feuille de route

Faites d'abord fonctionner la version en mémoire (sections 12.2 à 12.4), vérifiez l'UI, puis branchez Room. Cette progression « marche d'abord, persiste ensuite » est exactement la bonne méthode pour votre projet de groupe.

12.6 Exercices d'extension

32. Ajouter des catégories de tâches (filtrage par catégorie).
33. Afficher un compteur « tâches restantes ».
34. Permettre de modifier le libellé d'une tâche existante.
35. Ajouter une animation lors de l'ajout/suppression (`Modifier.animateItem`).
36. Écrire un test unitaire du `TodoViewModel` (ajout, bascule, suppression).

Partie 13 — Sécurité, données personnelles & bonnes pratiques

Une application manipule souvent des données sensibles. Concevoir « sécurisé » et respecter la vie privée n'est pas une option : c'est une responsabilité professionnelle et un critère de qualité. Cette partie présente les bons réflexes accessibles à un projet étudiant.

13.1 Le stockage des données

Choisissez le bon support selon la nature de la donnée :

Besoin	Solution recommandée
Préférences / petits réglages	DataStore (remplace SharedPreferences)
Données structurées locales	Room (base SQLite)
Fichiers (images, exports)	Stockage interne de l'application
Données distantes	API + cache local (offline-first)
Secrets (jetons, mots de passe)	EncryptedSharedPreferences / Keystore Android

Ne stockez jamais un secret en clair

Les clés d'API, jetons et mots de passe ne doivent pas être écrits en dur dans le code ni stockés en clair. Utilisez le **Keystore Android** ou un stockage chiffré, et ne committez jamais de secret dans Git.

13.2 Les communications réseau

- **HTTPS uniquement** : toutes les requêtes doivent passer par une connexion chiffrée (TLS). Android bloque le trafic en clair par défaut.
- **Valider les entrées** : ne faites jamais confiance aux données reçues du réseau ou saisies par l'utilisateur ; vérifiez-les.
- **Limiter les permissions** : ne demandez que ce dont l'application a réellement besoin.

13.3 La protection de la vie privée

Si votre application collecte des données personnelles (nom, e-mail, localisation), vous engagez votre responsabilité. Quelques principes inspirés du **RGPD** :

- **Minimisation** : ne collectez que les données strictement nécessaires.
- **Transparence** : expliquez clairement quelles données sont collectées et pourquoi (politique de confidentialité).
- **Consentement** : demandez l'accord de l'utilisateur pour les usages non essentiels.
- **Droit à l'effacement** : permettez à l'utilisateur de supprimer ses données.

Une exigence du Play Store

Toute application publiée doit fournir une politique de confidentialité et déclarer les données collectées (section « Sécurité des données »). Anticiper ces aspects dès la conception évite de tout reprendre à la fin.

13.4 Robustesse et gestion des erreurs

Une application sûre est aussi une application qui ne se met jamais dans un état incohérent :

- Gérez explicitement les cas d'échec (réseau coupé, données absentes, saisie invalide).
- Affichez des messages d'erreur clairs plutôt que de planter.
- Encapsulez les résultats (type `Result`) pour rendre les échecs visibles dans le code.
- Testez les cas limites (zéro, valeurs négatives, listes vides, textes très longs).

13.5 Checklist sécurité du projet

- Aucune clé/secret en clair dans le code ou dans Git.
- Communications en HTTPS uniquement.
- Permissions minimales, demandées au bon moment et expliquées.
- Données sensibles stockées de façon chiffrée.
- Entrées utilisateur et réponses réseau validées.
- Politique de confidentialité prête si des données personnelles sont collectées.

Partie 14 — Accessibilité & internationalisation

Une application de qualité est utilisable par **tout le monde**, partout. L'accessibilité (a11y) la rend exploitable par les personnes en situation de handicap ; l'internationalisation (i18n) l'adapte aux langues et régions. Ces deux dimensions valorisent fortement un projet.

14.1 Les principes de l'accessibilité

Quatre piliers (norme WCAG) guident une interface accessible :

- **Perceptible** : contrastes suffisants, alternatives textuelles aux images, taille de texte ajustable.
- **Utilisable** : cibles tactiles assez grandes, navigation possible au clavier/commutateur.
- **Compréhensible** : libellés clairs, comportement prévisible, messages d'erreur explicites.
- **Robuste** : compatible avec les technologies d'assistance (lecteur d'écran TalkBack).

14.2 L'accessibilité en Compose

```
// Description pour le lecteur d'écran
Icon(Icons.Default.Search, contentDescription = "Rechercher")

// Image décorative : pas de description
Image(painter = painterResource(R.drawable.deco), contentDescription = null)

// Regrouper et décrire un ensemble
Row(Modifier.semantics(mergeDescendants = true) {
    contentDescription = "Note : 4 étoiles sur 5"
}) { /* 5 icônes étoile */ }

// Taille tactile minimale (48 dp)
IconButton(onClick = { }, modifier = Modifier.minimumInteractiveComponentSize()) {
    Icon(Icons.Default.Add, contentDescription = "Ajouter")
}
```

Bonne pratique	Pourquoi
contentDescription pertinent	Le lecteur d'écran annonce l'élément
Contraste texte $\geq 4,5:1$	Lisibilité pour tous, y compris malvoyants
Cibles tactiles ≥ 48 dp	Faciles à atteindre, y compris en situation de handicap moteur
Ne pas coder l'information que par la couleur	Daltonisme : ajouter icône/texte
Respecter la taille de police système	Utiliser sp et non dp pour le texte

Tester l'accessibilité

Activez **TalkBack** sur un appareil et parcourez votre application les yeux fermés : vous repérerez vite les éléments non décrits. L'outil **Accessibility Scanner** signale automatiquement contrastes et cibles trop petites.

14.3 L'internationalisation (i18n)

Internationaliser, c'est concevoir l'application pour qu'elle s'adapte à plusieurs langues et régions sans modifier le code. La clé : **externaliser tous les textes** dans les ressources.

```
<!-- res/values/strings.xml (français par défaut) -->
<string name="bienvenue">Bienvenue</string>

<!-- res/values-en/strings.xml (anglais) -->
<string name="bienvenue">Welcome</string>

// Dans le code Compose : on référence la ressource, jamais le texte en dur
Text(text = stringResource(R.string.bienvenue))
```

- Android choisit automatiquement le bon [strings.xml](#) selon la langue du téléphone.
- Gérer le pluriel avec [plurals](#), les formats de date/nombre selon la région.
- Prévoir les langues **droite-à-gauche** (arabe, hébreu) : Compose gère le sens via [LayoutDirection](#).

Réflexe dès le départ

Externaliser les chaînes coûte peu au début et énormément à la fin. Prenez l'habitude d'écrire chaque texte dans strings.xml : c'est un marqueur de code professionnel, même si vous ne traduisez pas tout de suite.

Partie 15 — Performance & optimisation

Une application fluide retient ses utilisateurs ; une application qui rame est désinstallée. Cette partie approfondit la performance en Compose et donne une méthode pour optimiser sans tâtonner.

15.1 Comprendre la fluidité (jank)

L'écran se rafraîchit 60 fois (ou plus) par seconde. Pour rester fluide, chaque image doit être produite en moins de ~16 ms. Quand une image dépasse ce budget, l'utilisateur perçoit une **saccade** (jank). Les causes fréquentes : trop de recompositions, travail lourd sur le thread principal, listes mal optimisées.

15.2 Maîtriser les recompositions

- **Stabilité des types** : préférez des `data class` immuables et des collections immuables (`kotlinx.collections.immutable`). Une liste mutable rend un composable « instable » et provoque des recompositions inutiles.
- **Clés stables** dans les listes Lazy (`key = { it.id }`).
- **derivedStateOf** pour les calculs dérivés coûteux, afin de ne pas recomposer à chaque frappe.
- **remember** les objets et calculs coûteux pour ne pas les refaire à chaque recomposition.
- **Lambdas stables** : éviter de recréer des lambdas à chaque appel quand c'est évitable.

```
// derivedStateOf : ne recompose le bouton « haut de page »
// que lorsque la condition (et non l'offset) change réellement.
val afficherBouton by remember {
    derivedStateOf { listState.firstVisibleItemIndex > 5 }
}
```

15.3 Différer la lecture de l'état

Lire un état le plus tard possible limite l'étendue d'une recomposition. Pour une animation de position, utilisez la version lambda du modifier :

```
// ❌ lit l'état pendant la Composition → recompose tout
Box(Modifier.offset(x = decalage.dp))

// ✅ lit l'état pendant le Layout → saute la Composition
Box(Modifier.offset { IntOffset(decalage.roundToInt(), 0) })
```

15.4 Optimiser les listes

- Utiliser `LazyColumn/LazyRow` (jamais une `Column` défilante pour de longues listes).
- Fournir des `key` stables et un `contentType` si les éléments sont hétérogènes.
- Éviter les calculs lourds dans l'élément de liste ; les préparer en amont dans le `ViewModel`.

15.5 Mesurer : la méthode

37. Reproduire le ralentissement avec le **Layout Inspector** (compteur de recompositions) ou le **Profiler**.

38. Identifier la cause (composable instable, travail bloquant, requête synchrone).

39. Corriger une seule chose, puis **re-mesurer**.

40. Générer un **Baseline Profile** pour accélérer démarrage et défilement en production.

Ne pas optimiser à l'aveugle

L'optimisation prématurée complique le code sans bénéfice prouvé. Mesurez d'abord, corrigez le vrai goulot d'étranglement, et vérifiez le gain. La clarté du code prime tant qu'il n'y a pas de problème mesuré.

Partie 16 — Bonnes pratiques d'ingénierie & qualité du code

La qualité du code est l'un des critères majeurs du projet. Un code propre est lisible, testable, et facile à faire évoluer en équipe. Cette partie rassemble des principes concrets, applicables immédiatement.

16.1 Nommer et structurer

- **Noms explicites** : `nombreDeTaches` plutôt que `n` ; `calculerTotal()` plutôt que `calc()`.
- **Fonctions courtes** : une fonction = une responsabilité. Si elle dépasse l'écran, découpez-la.
- **Éviter la duplication (DRY)** : factorisez le code répété dans une fonction réutilisable.
- **Constantes nommées** : remplacez les « nombres magiques » par des constantes claires.
- **Cohérence** : appliquez partout le style Kotlin officiel (reformatage automatique).

16.2 Les principes SOLID (en bref)

Cinq principes d'orientation objet qui guident une bonne conception, applicables même à un projet étudiant :

Principe	Idée
S · Responsabilité unique	Une classe/fonction n'a qu'une seule raison de changer
O · Ouvert/fermé	Ouvert à l'extension, fermé à la modification
L · Substitution de Liskov	Une implémentation doit pouvoir remplacer son interface
I · Ségrégation d'interfaces	Des interfaces petites et ciblées plutôt qu'une énorme
D · Inversion de dépendances	Dépendre d'abstractions (interfaces), pas d'implémentations

En pratique dans votre projet

L'architecture UI / ViewModel / Repository applique déjà SOLID : chaque couche a une responsabilité unique (S), et le ViewModel dépend d'une [interface Repository](#) et non d'une implémentation concrète (D). C'est ce qui rend le code testable.

16.3 Reconnaître les « code smells »

Un **code smell** est un indice qu'un passage mériterait d'être amélioré (refactorisé). Quelques signaux à surveiller :

Signal	Remède
Fonction très longue	Découper en sous-fonctions nommées
Code dupliqué	Extraire une fonction commune
Trop de paramètres	Regrouper dans une data class
Commentaire qui explique un code obscur	Réécrire le code pour qu'il se passe de commentaire

Signal	Remède
Logique métier dans l'UI	Déplacer dans le ViewModel
Valeurs codées en dur partout	Centraliser dans des constantes/ressources

Refactoriser en sécurité

Refactorisez par petites étapes, en vous appuyant sur les outils d'Android Studio (renommage, extraction) et sur vos tests : ils garantissent que le comportement ne change pas. Améliorer le code en continu coûte bien moins cher qu'une grande réécriture.

16.4 La revue de code en équipe

Avant de fusionner une contribution dans `main`, un coéquipier la **relit**. La revue de code améliore la qualité et diffuse la connaissance dans l'équipe.

- Le code fait-il ce qu'il doit, simplement ?
- Est-il lisible et bien nommé ?
- Y a-t-il des cas d'erreur non gérés ?
- Des tests couvrent-ils le changement ?
- Respecte-t-il l'architecture et le style du projet ?

16.5 Conventions de commit

Un historique Git clair est une documentation vivante. Adoptez une convention simple, par exemple **Conventional Commits** :

```
feat: ajoute l'écran d'historique des calculs
fix: corrige la division par zéro
refactor: extrait le moteur de calcul dans Evaluateur
docs: complète le README
test: ajoute les tests du ViewModel
```

Petits commits, messages clairs

Committez souvent, avec un message à l'impératif décrivant l'intention. Un bon historique facilite la relecture, le retour en arrière et la compréhension du projet par l'enseignant.

Partie 17 — Étude de cas guidée : application Météo (API REST)

Cette troisième étude de cas illustre une application **connectée** : elle interroge une **API REST** sur Internet, met les données en cache localement et les affiche. C'est le squelette idéal pour tout projet « qui va chercher des données en ligne » (actualités, films, sport, etc.).

17.1 Architecture de l'application

On retrouve les couches habituelles : un **ApiService** (Retrofit) pour le réseau, un **Dao** (Room) pour le cache, un **Repository** qui combine les deux (offline-first), un **ViewModel** pour l'état, et l'UI Compose.

Couche	Composant
Réseau	MeteoApi (Retrofit) + MeteoDto
Local	MeteoDao + MeteoEntity (Room)
Données	MeteoRepository (offline-first)
Présentation	MeteoViewModel + MeteoUiState
UI	MeteoEcran (Compose)

17.2 La couche réseau (Retrofit)

```
data class MeteoDto(val temperature: Double, val description: String)

interface MeteoApi {
    @GET("weather")
    suspend fun getMeteo(
        @Query("q") ville: String,
        @Query("appid") cle: String,
        @Query("units") unites: String = "metric",
        @Query("lang") langue: String = "fr"
    ): MeteoDto
}
```

17.3 Le cache local (Room)

```
@Entity(tableName = "meteo")
data class MeteoEntity(
    @PrimaryKey val ville: String,
    val temperature: Double,
    val description: String,
    val majA: Long // horodatage de la dernière mise à jour
)

@Dao
interface MeteoDao {
    @Query("SELECT * FROM meteo WHERE ville = :ville")
    fun observer(ville: String): Flow<MeteoEntity>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun enregistrer(meteo: MeteoEntity)
```

}

17.4 Le Repository (offline-first)

```
class MeteoRepository(
    private val api: MeteoApi,
    private val dao: MeteoDao,
    private val cle: String
) {
    // L'UI observe le cache local
    fun observer(ville: String): Flow<Meteo?> =
        dao.observer(ville).map { it?.toModel() }

    // On rafraîchit depuis le réseau, puis on écrit en base
    suspend fun rafraichir(ville: String) {
        val dto = api.getMeteo(ville, cle)
        dao.enregistrer(MeteoEntity(ville, dto.temperature, dto.description,
            System.currentTimeMillis()))
    }
}
```

17.5 Le ViewModel

```
data class MeteoUiState(
    val ville: String = "Lomé",
    val meteo: Meteo? = null,
    val chargement: Boolean = false,
    val erreur: String? = null
)

class MeteoViewModel(private val repo: MeteoRepository) : ViewModel() {
    var state by mutableStateOf(MeteoUiState()); private set

    init { observerVille("Lomé"); rafraichir() }

    private fun observerVille(ville: String) {
        viewModelScope.launch {
            repo.observer(ville).collect { meteo ->
                state = state.copy(meteo = meteo)
            }
        }
    }

    fun rafraichir() {
        viewModelScope.launch {
            state = state.copy(chargement = true, erreur = null)
            try { repo.rafraichir(state.ville) }
            catch (e: Exception) { state = state.copy(erreur = "Réseau indisponible") }
            finally { state = state.copy(chargement = false) }
        }
    }
}
```

17.6 L'interface (Compose)

@Composable

```
fun MeteoEcran(viewModel: MeteoViewModel) {  
    val state = viewModel.state  
    Column(Modifier.fillMaxSize().padding(24.dp),  
        horizontalAlignment = Alignment.CenterHorizontally) {  
  
        Text(state.ville, style = MaterialTheme.typography.headlineMedium)  
  
        when {  
            state.chargement && state.meteo == null -> CircularProgressIndicator()  
            state.erreur != null && state.meteo == null ->  
                Text(state.erreur!!, color = MaterialTheme.colorScheme.error)  
            state.meteo != null -> {  
                Text("${state.meteo!!.temperature.roundToInt()} °C",  
                    style = MaterialTheme.typography.displayMedium)  
                Text(state.meteo!!.description)  
            }  
        }  
  
        Spacer(Modifier.height(16.dp))  
        Button(onClick = { viewModel.rafraichir() }) { Text("Actualiser") }  
    }  
}
```

Ce que ce cas enseigne

Le patron offline-first : l'UI lit toujours le cache local (toujours quelque chose à afficher, même hors-ligne), et le réseau ne fait que rafraîchir la base. C'est l'architecture de référence pour une application connectée robuste.

17.7 Étapes pour le réaliser

41. Obtenir une clé d'API gratuite (ex. service météo public) et l'utiliser en respectant l'annexe sécurité (ne pas la committer en clair).
42. Ajouter les dépendances Retrofit, le convertisseur JSON et Room.
43. Déclarer la permission [INTERNET](#) dans le manifeste.
44. Implémenter les couches dans l'ordre : réseau, local, repository, ViewModel, UI.
45. Gérer les états : chargement, erreur, succès, hors-ligne.

Partie 18 — Recettes pratiques (FAQ technique)

Cette partie rassemble des « recettes » : des solutions courtes à des besoins récurrents que vous rencontrerez dans votre projet. Gardez-la à portée de main.

18.1 Afficher un message temporaire (Snackbar)

```
val snackbarHostState = remember { SnackbarHostState() }
val scope = rememberCoroutineScope()
Scaffold(snackbarHost = { SnackbarHost(snackbarHostState) }) { padding ->
    Button(onClick = {
        scope.launch { snackbarHostState.showSnackbar("Enregistré !") }
    }, modifier = Modifier.padding(padding)) { Text("Enregistrer") }
}
```

18.2 Demander confirmation avant de supprimer

```
var confirmer by remember { mutableStateOf(false) }
if (confirmer) {
    AlertDialog(
        onDismissRequest = { confirmer = false },
        confirmButton = { TextButton(onClick = { supprimer(); confirmer = false }) {
            Text("Supprimer") } },
        dismissButton = { TextButton(onClick = { confirmer = false }) { Text("Annuler") } },
        title = { Text("Confirmer") }, text = { Text("Supprimer cet élément ?") }
    )
}
```

18.3 Ouvrir un lien ou envoyer un e-mail

```
val context = LocalContext.current
// Lien web
context.startActivity(Intent(Intent.ACTION_VIEW, Uri.parse("https://exemple.com")))
// E-mail
context.startActivity(Intent(Intent.ACTION_SENDTO, Uri.parse("mailto:contact@exemple.com")))
```

18.4 Formater une date ou un nombre

```
val date = SimpleDateFormat("dd/MM/yyyy", Locale.FRANCE).format(Date())
val prix = NumberFormat.getCurrencyInstance(Locale.FRANCE).format(19.9) // 19,90 €
```

18.5 Réagir à la saisie avec un délai (recherche)

```
var requete by remember { mutableStateOf("") }
LaunchedEffect(requete) {
    delay(300) // attend une pause de frappe
    if (requete.isNotBlank()) viewModel.rechercher(requete)
}
OutlinedTextField(value = requete, onValueChange = { requete = it },
    label = { Text("Rechercher") })
```

18.6 Afficher un état de chargement plein écran

```
Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
```

```
    if (state.chargement) CircularProgressIndicator()  
    else Contenu(state)  
}
```

18.7 Gérer le bouton « retour »

```
// Intercepter le retour système (ex. demander confirmation de sortie)  
BackHandler(enabled = modificationsNonEnregistrees) {  
    afficherDialogConfirmation = true  
}
```

18.8 Adapter à l'orientation / la taille d'écran

```
BoxWithConstraints {  
    if (maxWidth < 600.dp) ColonneUnique() // téléphone  
    else DeuxColonnes() // tablette / paysage  
}
```

Construire sa propre bibliothèque de recettes

Au fil du projet, notez les solutions que vous trouvez à des problèmes récurrents. Cette « boîte à outils » personnelle accélère énormément le développement et se partage facilement dans l'équipe.

Partie 19 — Conception d'interface : maquettage & Material Design

Avant de coder un écran, il faut le **concevoir**. Une bonne conception fait gagner un temps considérable et améliore nettement l'expérience utilisateur — un critère important du projet. Cette partie présente la démarche et les principes de design.

19.1 Du besoin à la maquette

La conception part des **besoins** (user stories) et les traduit en écrans. On procède par niveaux de précision croissante :

Étape	Description
Croquis (sketch)	Esquisses rapides à main levée des écrans et de leur enchaînement
Wireframe	Maquette en niveaux de gris : structure et disposition, sans détail visuel
Maquette (mockup)	Version avec couleurs, typographie et composants réels
Prototype	Maquette cliquable simulant la navigation

Maquetter coûte moins cher que coder

Corriger un écran sur une maquette prend quelques minutes ; le refaire en code prend des heures. Esquissez et faites valider vos écrans avant de développer : c'est la marque d'une bonne méthode.

19.2 Les principes de Material Design 3

Material Design est le système de design de Google. Sa version 3 (**Material You**) met l'accent sur la personnalisation et l'accessibilité. Ses principes directeurs :

- **Hiérarchie** : guider l'œil par la taille, le contraste et l'espacement.
- **Cohérence** : mêmes composants et mêmes règles partout dans l'application.
- **Élévation** : les surfaces (cartes, barres) se distinguent par des ombres et des teintes.
- **Mouvement utile** : les transitions expliquent les changements d'état, sans distraire.
- **Accessibilité** : contrastes, tailles et cibles tactiles conformes par défaut.

19.3 Couleurs et typographie

Material 3 repose sur un **schéma de couleurs** dérivé d'une couleur principale, décliné en rôles sémantiques (primary, secondary, surface, error...) et en versions claire/sombre. La **typographie** suit une échelle (display, headline, title, body, label) qui structure le texte.

Rôle de couleur	Usage
primary / onPrimary	Éléments d'action principaux (boutons, accents)

Rôle de couleur	Usage
secondary	Accents secondaires
surface / background	Fonds des surfaces et de l'écran
error / onError	États d'erreur
outline	Bordures et séparateurs

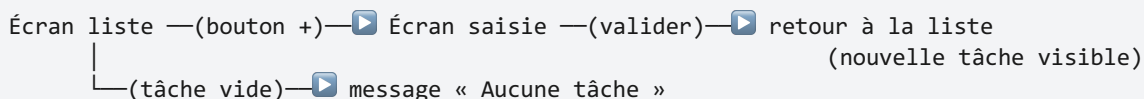
Un thème, pas des couleurs éparpillées

Définissez le schéma une fois (voir Partie 4) et référencez les rôles (`MaterialTheme.colorScheme.primary`) partout. L'application gagne en cohérence et s'adapte automatiquement au mode sombre.

19.4 Concevoir un parcours utilisateur

Un **parcours utilisateur** (user flow) décrit la suite d'écrans et d'actions pour accomplir une tâche. Le dessiner révèle les étapes inutiles et les impasses.

Exemple de parcours « ajouter une tâche » :



Pour chaque écran, prévoyez les **états** : vide, chargement, succès, erreur. Une application qui gère ces quatre cas paraît immédiatement professionnelle.

19.5 Outils de conception

- **Papier et crayon** : le plus rapide pour les premières esquisses, idéal en équipe.
- **Figma** : outil de maquettage collaboratif gratuit, très répandu, avec des kits Material 3.
- **Excalidraw / draw.io** : croquis et schémas de parcours rapides.

Concevoir en équipe

Réunissez l'équipe autour des maquettes avant de coder : tout le monde partage la même vision du produit, on répartit mieux les écrans, et on évite de développer deux fois la même chose. La conception est un travail collectif.

Partie 20 — Synthèse & fiches récapitulatives

Cette partie rassemble, sous forme de fiches, les idées essentielles du support. À relire avant un TP, avant un jalon de projet, ou avant la soutenance.

20.1 Fiche — Les piliers de Jetpack Compose

Notion	À retenir
Déclaratif	On décrit l'UI pour un état ; le framework met à jour l'affichage
Composable	Fonction @Composable qui émet de l'UI ; nom en majuscule
État	remember + mutableStateOf ; rememberSaveable pour survivre à la rotation
Recomposition	Ré-exécution ciblée quand l'état lu change
State hoisting	Remonter l'état : composables stateless, réutilisables et testables
UDF	État descendant, événements remontants
Effets	LaunchedEffect/DisposableEffect pour les effets de bord

20.2 Fiche — L'architecture recommandée

Couche	Rôle	Composants
UI	Afficher l'état, émettre des événements	Composables
Présentation	Détenir l'état, orchestrer la logique	ViewModel, UiState
Données	Fournir et persister les données	Repository, Retrofit, Room

20.3 Fiche — Le cycle de vie d'un projet

Phase	Question clé
Analyse	Quel problème, pour quels utilisateurs ?
Conception	Quels écrans, quelle architecture, quelles données ?
Développement	Comment implémenter, fonctionnalité par fonctionnalité ?
Tests	Le comportement est-il correct et robuste ?
Déploiement	Comment produire et distribuer un livrable signé ?
Suivi	Comment mesurer, corriger et améliorer ?

20.4 Fiche — Réflexes de qualité

- Petits composables stateless ; logique dans le ViewModel.
- Types immuables et clés stables pour la performance.
- Textes externalisés, thème centralisé, accessibilité de base.
- Gestion explicite des états vide / chargement / succès / erreur.

- Tests de la logique métier ; Git régulier ; documentation continue.

20.5 Fiche — Du cours au projet

Le chemin le plus sûr vers une bonne note : **commencer tôt**, livrer une fonctionnalité de bout en bout rapidement, capitaliser sur les trois études de cas (calculatrice, To-Do, météo), tester en continu, soigner l'interface en fin de parcours, et préparer une démonstration claire. La régularité et la méthode comptent autant que la technique.

En une phrase

Une application mobile réussie, c'est un besoin bien compris, une architecture claire, un code testé, une interface soignée et un travail d'équipe bien organisé — exactement ce que ce projet vous apprend à conduire.

Annexes

Annexe A — Aide-mémoire des modifieurs courants

Modifieur	Effet
size / width / height	Fixe les dimensions
fillMaxSize / fillMaxWidth	Occupe tout l'espace disponible
padding	Marge (externe ou interne selon la position dans la chaîne)
background	Couleur ou dégradé de fond
clickable	Rend l'élément cliquable (avec effet ripple)
clip	Découpe selon une forme (RoundedCornerShape...)
weight (dans Row/Column)	Répartit l'espace restant proportionnellement
align / wrapContentSize	Aligne l'élément dans son parent
border	Bordure colorée
offset	Décale la position sans affecter la mise en page
testTag	Étiquette pour les tests d'UI

Annexe B — Erreurs fréquentes et solutions

Symptôme	Cause probable / Solution
L'état ne se met pas à jour	Variable normale au lieu de mutableStateOf, ou oubli de remember
L'état est perdu après rotation	Utiliser rememberSaveable ou le ViewModel
L'UI passe sous la barre du haut	innerPadding du Scaffold non appliqué
Boucle de recomposition / requête répétée	Effet de bord hors de LaunchedEffect
Liste lente, animations cassées	Absence de key stable dans items()
Recompositions superflues	Type instable (List mutable) ; viser l'immuabilité
Message/navigation qui se répète	Événement modélisé comme un état au lieu d'un Channel

Annexe C — Le cycle de vie d'une composition

Comprendre le cycle de vie aide à placer correctement état et effets :

1. **Composition initiale** : le composable s'exécute pour la première fois et émet son UI.
2. **Recomposition** : déclenchée quand un état lu par le composable change ; seuls les composables concernés se ré-exécutent.

3. **Sortie de composition** : le composable quitte l'arbre ; `DisposableEffect` libère alors ses ressources.

Les fonctions `remember` vivent entre la composition initiale et la sortie. Les `LaunchedEffect` sont relancés quand leur clé change et annulés à la sortie. Penser en termes de ce cycle évite fuites mémoire et bugs subtils.

Annexe D — Ressources officielles pour continuer

- **Documentation Compose** : developer.android.com/develop/ui/compose
- **Codelabs Compose** : parcours pratiques guidés, du débutant à l'avancé.
- **Now in Android** : application de référence open-source illustrant l'architecture recommandée.
- **Compose Material 3** : m3.material.io — lignes directrices de design.
- **Notes de version** : suivez le BOM et le blog des développeurs Android pour les nouveautés.

Annexe E — Lexique du développement mobile

Terme	Définition
SDK	Software Development Kit : ensemble d'outils pour développer sur une plateforme
IDE	Environnement de développement intégré (ici Android Studio)
API level	Numéro de version d'Android visé (ex. 34)
APK / AAB	Formats de paquet Android : installable / bundle de publication
AVD	Android Virtual Device : appareil émulé
Gradle	Système de build qui compile l'application
ADB	Android Debug Bridge : pont de communication avec l'appareil
UI / UX	Interface utilisateur / Expérience utilisateur
MVVM	Model-View-ViewModel : patron d'architecture
ViewModel	Composant détenant l'état de l'écran, survivant aux rotations
Coroutine	Mécanisme d'exécution asynchrone non bloquante
Recomposition	Ré-exécution d'un composable quand son état change
Backlog / Sprint	Liste priorisée de tâches / itération courte de développement
MVP	Minimum Viable Product : version minimale fonctionnelle
Crash / ANR	Plantage / « Application Not Responding » (UI figée)

Annexe F — FAQ du débutant

« Mon application plante au démarrage, que faire ? »

Ouvrez [Logcat](#), filtrez par niveau [Error](#), et lisez la première ligne de la pile d'appels qui mentionne votre package : elle indique le fichier et la ligne fautive.

« **Gradle échoue / le code est tout rouge.** »

Lancez [Sync Now](#). Si le problème persiste : [File](#) → [Invalidate Caches / Restart](#), puis [Build](#) → [Clean Project](#) et [Rebuild](#).

« **Mon état se réinitialise quand je tourne l'écran.** »

Utilisez [rememberSaveable](#) pour un état local simple, ou déplacez l'état dans un [ViewModel](#) (recommandé).

« **L'émulateur est très lent.** »

Activez l'accélération matérielle, choisissez une image système adaptée à votre processeur, ou testez sur un téléphone physique en débogage USB.

« **Comment partager le code avec mon équipe ?** »

Créez un dépôt Git (GitHub/GitLab), ajoutez un [.gitignore](#) Android, et committez régulièrement sur des branches de fonctionnalité.

« **Par où commencer mon projet ?** »

Rédigez le cahier des charges, dessinez les maquettes, créez le projet, mettez en place la navigation et l'architecture, puis implémentez une fonctionnalité de bout en bout avant d'élargir.

Annexe G — Modèle de cahier des charges

Modèle à compléter par chaque équipe en début de projet (un cahier des charges tient généralement en 2 à 4 pages).

4. **Titre du projet et équipe** : nom de l'application, membres et rôles.
5. **Contexte et problème** : quel besoin l'application résout-elle ?
6. **Public cible** : qui sont les utilisateurs, dans quel contexte d'usage ?
7. **Objectifs** : ce que l'application doit permettre de faire (3 à 5 objectifs).
8. **Fonctionnalités indispensables** : la liste du « minimum viable ».
9. **Fonctionnalités optionnelles** : à réaliser si le temps le permet.
10. **User stories** : besoins formulés côté utilisateur + critères d'acceptation.
11. **Contraintes techniques** : Android natif, minSdk, données locales/réseau, bibliothèques.
12. **Maquettes** : esquisses des écrans principaux.
13. **Planning et jalons** : échéances et répartition des tâches.

Annexe H — Plan type du rapport de projet

14. **Introduction** : contexte, objectif, présentation de l'équipe.
15. **Analyse des besoins** : cahier des charges, user stories.
16. **Conception** : architecture, maquettes, modèle de données, choix techniques.

17. **Réalisation** : fonctionnalités développées, difficultés rencontrées et solutions.

18. **Tests** : stratégie de test, résultats, bugs corrigés.

19. **Gestion de projet** : organisation, outils, planning réel vs prévu.

20. **Bilan** : ce qui a été réalisé, limites, perspectives d'amélioration.

21. **Annexes** : captures d'écran, lien du dépôt Git, manuel d'utilisation.

Annexe I — Commandes Gradle & ADB utiles

Commande	Effet
./gradlew assembleDebug	Compiler un APK de debug
./gradlew assembleRelease	Compiler un APK de release (à signer)
./gradlew bundleRelease	Compiler un AAB pour le Play Store
./gradlew test	Lancer les tests unitaires locaux
./gradlew connectedAndroidTest	Lancer les tests instrumentés (appareil requis)
./gradlew clean	Nettoyer les fichiers de build
adb devices	Lister les appareils connectés
adb install app-debug.apk	Installer un APK sur l'appareil
adb logcat	Afficher les journaux de l'appareil

Annexe J — Checklist de fin de projet

À parcourir avant le rendu et la soutenance :

- L'application répond au cahier des charges et ne plante pas.
- Les états vide / chargement / succès / erreur sont gérés sur chaque écran.
- Le code est organisé (UI / ViewModel / Données) et lisible.
- Aucun secret en clair ; permissions minimales ; HTTPS.
- Des tests couvrent la logique métier et un écran clé.
- Le dépôt Git est à jour, avec un historique propre et un .gitignore.
- L'APK signé est généré et installable.
- README, manuel d'utilisation et rapport sont rédigés.
- La démonstration est préparée et répétée.

Annexe K — Grille d'auto-évaluation de l'équipe

Notez chaque critère de 1 (à améliorer) à 4 (excellent) avant la soutenance :

Critère	1	2	3	4
Conformité au cahier des charges				
Stabilité (absence de plantage)				

Critère	1	2	3	4
Qualité et lisibilité du code				
Architecture (séparation des couches)				
Expérience utilisateur / accessibilité				
Tests				
Gestion de projet (Git, planning)				
Documentation				

S'auto-évaluer, c'est progresser

Remplir cette grille en équipe avant la soutenance révèle les points faibles à corriger en priorité et prépare les questions du jury. C'est un réflexe professionnel précieux.

Annexe L — Parcours d'apprentissage recommandé

Si vous débutez, voici un ordre de progression efficace pour devenir autonome :

- 22. **Bases** : Kotlin essentiel (Partie 1) + installation et premier projet (Partie 0).
- 23. **Interface** : composables, layouts, état, listes (Partie 2).
- 24. **Structure** : architecture, ViewModel, navigation (Partie 3).
- 25. **Mise en pratique** : refaire la calculatrice (Partie 7), puis le To-Do (Partie 12).
- 26. **Données** : Flow, Retrofit, Room, tests (Partie 5) ; refaire la météo (Partie 17).
- 27. **Polissage** : UI avancée, accessibilité, performance (Parties 4, 14, 15).
- 28. **Projet** : appliquer le tout au projet de groupe (Parties 8, 11, 16).

Apprendre par projet

La meilleure progression mêle théorie et pratique : après chaque notion, appliquez-la immédiatement dans une petite application. Les trois études de cas du support sont conçues pour cela.

Annexe M — Modèle de rétroplanning d'équipe

À adapter selon la durée réelle du projet (exemple sur 6 semaines, 3-4 membres) :

Semaine	Objectif	Responsable(s)
1	Cahier des charges, maquettes, dépôt Git, architecture	Toute l'équipe
2	Navigation + écrans vides + thème	Dév. + UX
3-4	Fonctionnalités indispensables (cœur)	Dév.
5	Données (réseau/local) + premiers tests	Dév. + Qualité
6	Finitions UI, APK, documentation, répétition de la démo	Toute l'équipe

Garder une marge

Prévoyez toujours un peu de marge en fin de planning : il y a toujours un imprévu. Mieux vaut finir les fonctionnalités indispensables une semaine avant la fin et consacrer le reste aux finitions et aux tests.

Annexe N — Mini-lexique de la syntaxe Kotlin

Élément	Signification
val / var	Variable immuable / mutable
fun	Déclaration de fonction
? (ex. String?)	Type pouvant être null
?. / ?:	Appel sécurisé / opérateur Elvis (valeur par défaut)
{ ... }	Lambda (fonction anonyme) ou corps de bloc
data class	Classe porteuse de données (equals, copy... générés)
sealed interface	Hiérarchie fermée (idéale pour modéliser un état)
object	Singleton / instance unique
suspend	Fonction suspendable (coroutine)
it	Argument implicite d'une lambda à un paramètre
when	Aiguillage multi-cas (expression)
by	Délégation de propriété (ex. remember, by viewModels())

Annexe O — Pour continuer après le projet

Une fois le projet terminé, plusieurs pistes permettent d'aller plus loin :

- Publier réellement l'application sur le Play Store (compte développeur).
- Explorer **Compose Multiplatform** pour viser iOS et le desktop avec le même code.
- Approfondir les tests, l'intégration continue et la performance (Baseline Profiles).
- Contribuer à un projet open-source pour pratiquer le travail en équipe à grande échelle.
- Suivre les nouveautés via le blog des développeurs Android et les Codelabs officiels.

Le mot de la fin

Le développement mobile s'apprend en construisant. Ce que vous réalisez dans ce projet — analyser, concevoir, coder, tester, livrer, présenter — constitue une expérience complète, directement valorisable dans la suite de votre parcours. Bon projet à toutes les équipes !

— Fin du support de cours —